Multimedia Retrieval

University of Basel Course Textbook

Multimedia Retrieval - 2026 - Uni Basel

Roger Weber

Preface

This textbook accompanies the Multimedia Retrieval course at the University of Basel. The online edition remains the authoritative source for interactive material and updates.

Contents

Multimedia Retrieval

About this book

This is the companion textbook for **Multimedia Retrieval**, a Master's-level course at the University of Basel, edition HS26. Course announcements, exercise submissions, and scheduling are managed through ADAM and GitHub

From Ctrl+F to AI Answers

Suppose you need to find all books about cats in a library. A simple string search (Ctrl+F for "cat") scans every page until it finds a match. Even for a modest collection this is impractical, and the result is merely a Boolean filter: it tells you which books contain the string "cat", but it cannot tell you which ones are *most relevant* to what you are looking for. For popular topics, you are left with a long list of matches, and have to manually browse through the results to find what you wanted.

Retrieval systems solve this problem in two steps. First, they organize information in advance through preprocessing and indexing: extracting features from documents, building data structures that enable fast lookup, and storing statistics that describe each document's content. Second, at query time they apply ranking models that score documents by their relevance to the query rather than simply filtering by presence or absence of a term.

Modern AI systems have made retrieval even more critical. Large language models can in theory process long documents, but doing so is expensive (costs scale with token count) and prone to hallucination when the model must rely on memorized knowledge. In practice, it is far more effective to first retrieve the relevant passages from a document collection and feed only those into the model's context. This pattern, retrieval-augmented generation, depends entirely on the quality of the retrieval step: if the wrong passages are retrieved, the generated answer will be wrong too.

The Semantic Gap

Consider searching for a picture of a cat. Nothing in the raw pixel values tells a machine that an image depicts a cat. The system sees a grid of numbers; the user asks a question about meaning. This mismatch between how humans describe what they want and how machines represent information is called the *semantic gap* (Figure 0.1).

Figure 0.1: A human sees a "cat"; a machine sees a grid of binary values.

The gap exists in text too, though it is narrower. A search for "cat" will match many documents containing that word or its plural "cats", but it will miss documents that discuss "felines" or mention specific breeds by name without ever using the word "cat". In images, audio, and video the gap is far wider: there is no surface-level token to match against.

The semantic gap plagued earlier retrieval systems, especially those working with non-text media. Recent breakthroughs in AI, particularly dense embeddings and transformer models, have dramatically narrowed it by learning to map both queries and documents into a shared space where meaning, not surface form,

determines similarity. This book traces the full path from systems that match keywords to systems that understand meaning.

Closing the Gap

The semantic gap is not one problem but a spectrum of problems, and this book attacks them in order of increasing sophistication.

The first step is keyword matching: building an index so that a search for "cat" instantly finds every document containing that word, without scanning the entire collection. This alone closes the gap for queries where the user's words appear verbatim in the target. But as we saw, many relevant documents use different vocabulary.

The second step is statistical ranking. Instead of treating every keyword match equally, we weight terms by how informative they are and score documents by how well they match the query overall. A document that mentions "cat" once in passing ranks lower than one dedicated to the topic. This narrows the gap further, but only within the space of shared vocabulary.

The third step is semantic matching. We represent both queries and documents as dense vectors in a shared space, trained so that "cat" and "feline" end up near each other. Now the system can find relevant documents even when no words overlap. This is where the gap narrows dramatically for text, and where non-text media (images, audio) first become searchable by meaning.

The final step is retrieval-augmented generation. Rather than returning a ranked list of documents, the system retrieves the most relevant passages and feeds them to a language model that synthesizes a direct answer. The gap between question and answer is closed end-to-end.

Each step builds on the previous one. A RAG pipeline still needs an index (step 1), still benefits from good ranking (step 2), and works best with semantic retrieval (step 3). This is why the book is structured as a cumulative progression rather than a menu of independent topics.

This progression was not designed in one stroke. It accumulated over six decades, as each generation of researchers added a new way to narrow the gap. The optional history below traces that path from the first computerized catalogues to today's generative systems.

From card catalogues to transformers: six decades of retrieval (optional reading)

The 1960s saw the birth of computerized information retrieval. Calvin Mooers had coined the term "information retrieval" in 1950, but serious research began only in the 1960s. H. P. Luhn created KWIC (Key Word In Context) indexes, IBM developed the STAIRS system, and Gerard Salton began work on SMART at Cornell. Most systems remained experimental because few texts existed in machine-readable form, and retrieval was still largely a human-mediated activity (Figure 0.2).

Figure 0.2: Library retrieval in the 1960s: a manual reference desk (left) beside an early terminal-based catalogue (right).

The 1970s brought the first practical systems. Widespread computer typesetting produced the machine-readable text needed for large-scale retrieval. Salton's vector space model represented documents and queries as vectors whose similarity could be measured mathematically. Robertson and Sparck Jones developed the probabilistic framework that would later lead to BM25. Commercial systems like Lockheed Dialog and MEDLARS demonstrated that automated retrieval could work at scale (Figure 0.3).

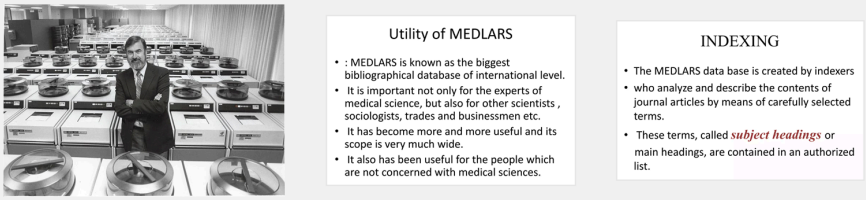


Figure 0.3: The MEDLARS system: an early large-scale bibliographic database running on reel-to-reel tape storage.

The 1980s refined these models. Boolean retrieval became the dominant commercial approach, letting users combine terms with AND, OR, and NOT (Figure 0.4). Robertson formalized the Probability Ranking Principle. The Okapi system at City University London gave its name to the BM25 ranking function. Latent Semantic Indexing appeared as a method to reveal hidden relationships between terms.

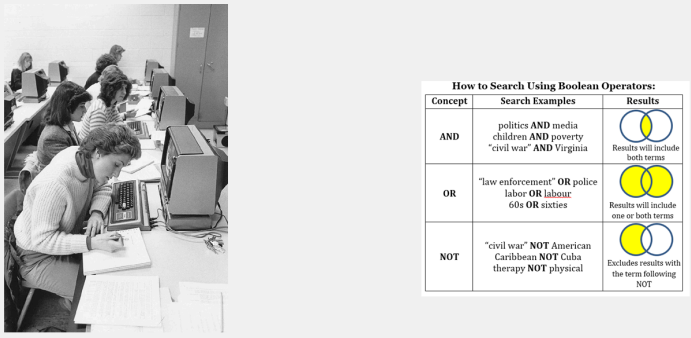


Figure 0.4: Boolean operators (AND, OR, NOT) as set operations that select matching documents.

The 1990s brought the web revolution. Early search engines (Archie, WebCrawler, AltaVista) appeared to help users navigate the new online world (Figure 0.5). The most significant development was PageRank (Page and Brin, 1996), which ranked pages by the authority of incoming links rather than keyword matching alone.

Figure 0.5: Early web interfaces circa 1998-1999, showing keyword search boxes and directory-style navigation.

The 2000s introduced machine learning into ranking. Learning to Rank (LTR) trained supervised models to order results by relevance. BM25 became a standard baseline. Deep learning re-emerged after a quiet period, driven by more powerful GPUs and larger datasets (Figure 0.6).

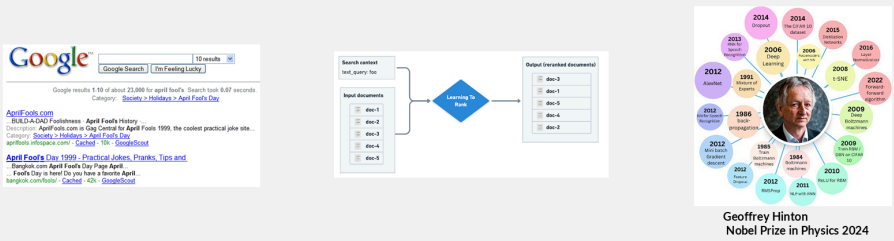


Figure 0.6: The 2000s: Google-era keyword search alongside the rise of Learning-to-Rank.

The 2010s saw deep learning applied directly to retrieval. Neural ranking models replaced handcrafted features with learned representations. Google introduced BERT in 2018, enabling search engines to understand context and word relationships. Transformer architectures fundamentally changed how machines handle sequential data (Figure 0.7).

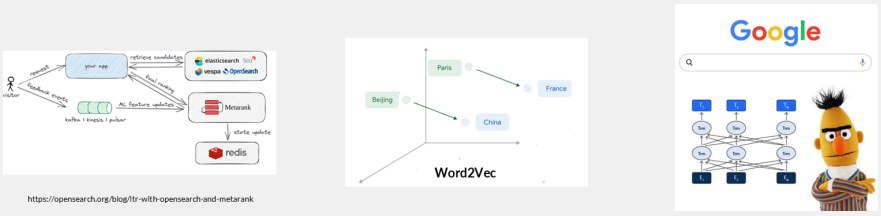


Figure 0.7: Neural ranking in the 2010s: a Learning-to-Rank pipeline (left), an embedding space (center), and contextual models like BERT (right).

The 2020s brought semantic search and vector databases as production infrastructure. Retrieval-Augmented Generation combines retrieval with large language models. Agentic systems can reason about information needs, execute multi-step retrieval, and synthesize results from multiple sources (Figure 0.8).

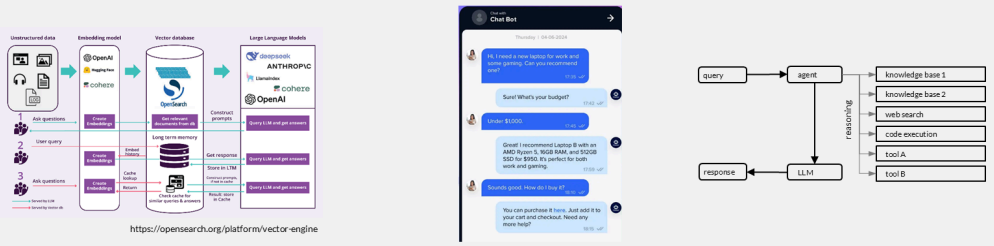


Figure 0.8: Three architectures for knowledge-grounded AI: a RAG pipeline with vector retrieval (left), a conversational interface (center), and an agentic system selecting among tools and knowledge sources (right).

How This Book Is Organized

The book is structured in three parts that build on each other. Each chapter focuses on one retrieval capability and the techniques to implement it. The path is cumulative: later chapters assume you have worked through the earlier ones.

Part I: Foundations (Chapters 1-3)

The first three chapters establish the fundamentals that every retrieval system relies on.

Chapter 1: Classical Text Retrieval introduces the core problem: given a query and a collection of documents, find the relevant ones. We build from Boolean retrieval (exact keyword matching) through TF-IDF weighting to BM25, the statistical ranking model that still powers production search engines today.

Chapter 2: Performance Evaluation asks the question every retrieval engineer must answer: does this system actually work? We cover precision, recall, ranked evaluation metrics (MAP, NDCG), and the experimental methodology for benchmarking retrieval systems.

Chapter 3: Advanced Text Processing looks at what happens before retrieval: how raw text is transformed into features that improve search quality. Tokenization, stemming, compound handling, query understanding, and intent classification.

Part II: Search Systems (Chapters 4-7)

The middle chapters move from individual techniques to complete systems.

Chapter 4: Index for Text Retrieval covers the data structures that make retrieval fast: inverted files, posting lists, compression, and how systems scale from thousands to billions of documents.

Chapter 5: Semantic Search introduces the shift from keyword matching to meaning matching. We trace the path from latent semantic indexing through word embeddings to transformer-based dense retrieval.

Chapter 6: Vector Search addresses the infrastructure challenge: once documents are represented as vectors, how do we find nearest neighbors efficiently among millions of embeddings? Approximate nearest neighbor algorithms, quantization, and vector databases.

Chapter 7: Retrieval-Augmented Generation combines retrieval with large language models. We cover chunking strategies, query transformation, retrieval pipelines, and how to build systems that generate answers grounded in retrieved evidence.

Part III: Advanced Topics (Chapters 8-12)

The final chapters extend retrieval beyond text.

Chapter 8: Web Search adds link analysis (PageRank, HITS) and web-specific ranking signals to the retrieval stack.

Chapter 9: Content Analysis covers structural and metadata-based features for multimedia documents.

Chapter 10: Visual Features applies retrieval to images: color histograms, texture descriptors, shape features, and modern deep visual features.

Chapter 11: Audio Features extends retrieval to audio: perceptual features, musical features, and fingerprinting for music recognition.

Chapter 12: Video Structural Features addresses the temporal dimension: shot detection, motion features, and how to search within video.

What to Expect in Each Chapter

Every chapter starts with a concrete scenario that motivates the problem, then develops the solution through four layers:

1. **Concepts and examples.** We introduce each technique through a running example with real data, so you can see what the method does before we formalize how it works.
2. **Formal foundations.** Key formulas and models are stated precisely, with plain-English intuition alongside the math. You will know both what to compute and why.
3. **Practical implementation.** Code examples show how techniques translate into working systems. Where relevant, we reference production tools (Lucene, FAISS, LangChain) so you can connect theory to practice.
4. **Hands-on notebooks.** Interactive demos let you run the techniques yourself, experiment with parameters, and observe how changes affect retrieval quality.
5. **Quiz questions.** Each chapter includes multiple-choice questions in the Quiz App to test your understanding and prepare for the exam.

Each chapter ends with a **summary** that recaps the most important concepts, formulas, and insights in condensed form, useful for review and exam preparation. Summaries also point to further reading for those who want to go deeper.

What You Will Be Able To Do

By the end of this book, you will be able to:

- Build a text retrieval system from scratch using an inverted index
- Evaluate retrieval quality with precision, recall, and ranking metrics
- Process raw text into features that improve search quality
- Design a semantic search system using dense embeddings
- Construct a RAG pipeline that answers questions from a document collection
- Apply retrieval techniques to non-text media: images, audio, and video
- Choose the right retrieval architecture for a given problem

We begin with the simplest useful system (keyword search over a small collection) and end with systems that search across media types and generate natural-language answers. Let us start.

Further Reading

These four sources trace the same four-step progression this book follows, from keyword matching to generation. Read them for depth beyond the chapter treatment.

- Manning, C. D., Raghavan, P., & Schütze, H. (2008). **Introduction to Information Retrieval**. *Cambridge University Press*. Free online edition. The standard reference for classical text retrieval: indexing, Boolean and vector-space models, and evaluation. It underpins Chapters 1 to 4.
- Robertson, S., & Zaragoza, H. (2009). **The Probabilistic Relevance Framework: BM25 and Beyond**. *Foundations and Trends in Information Retrieval*, 3(4), 333-389. PDF. The definitive account of BM25, the statistical ranking model that anchors the second step of the progression and still serves as the standard baseline.
- Lin, J., Nogueira, R., & Yates, A. (2021). **Pretrained Transformers for Text Ranking: BERT and Beyond**. *arXiv preprint*. arXiv:2010.06467. A survey of how transformer models moved retrieval from surface matching to meaning, the third step covered in Chapter 5.
- Lewis, P., Perez, E., Piktus, A., et al. (2020). **Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks**. *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2005.11401. The paper that introduced RAG, the final step where retrieval feeds a language model to generate grounded answers, developed in Chapter 7.

1 - Classical Text Retrieval

Every day, billions of queries hit search engines, email clients, and document archives. The core challenge is always the same: given a short query, find the most relevant items in a large collection. This chapter introduces the classical methods that solve this problem for text.

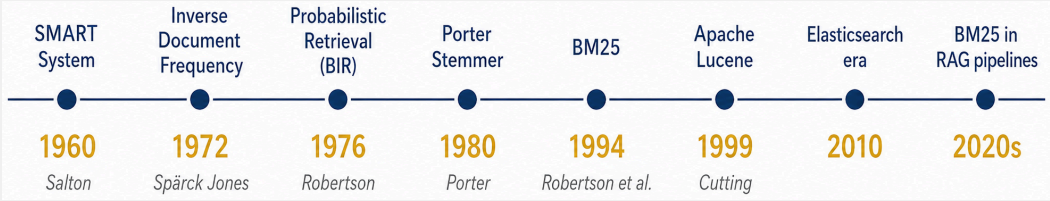
Text retrieval was one of the earliest problems in computer science to receive serious attention. Beginning in the 1950s, researchers like Gerard Salton and Karen Sparck Jones developed methods that worked remarkably well. The reason is fundamental: queries and documents share the same vocabulary. When a user types "climate change policy," the system can match those exact words in documents without needing to bridge different representations. This directness gave text retrieval a head start over image or audio search, where the gap between raw data and human meaning is far wider.

By the early 2000s, many considered text retrieval a solved problem. The core algorithms were mature, search engines worked well enough, and research attention shifted elsewhere. Recent developments in generative AI have changed the picture. Modern language models depend on precise, up-to-date text passages retrieved from external sources, and suddenly the quality of the retrieval stage matters more than ever. Classical text retrieval is no longer just the backbone of search engines. It is a critical component in modern AI systems.

We begin with Boolean filtering, the simplest approach, and trace the evolution toward ranked retrieval models that estimate how relevant each document is. Along the way, we develop the feature extraction pipeline that transforms raw text into searchable representations.

The milestones behind modern text search (optional reading)

Evolution of classical text retrieval: key milestones, methods, and contributors from the 1960s to the present:



- **1960 - SMART System (Salton):** Gerard Salton's SMART system at Cornell was the first to treat documents and queries as vectors in a term space. It established the experimental methodology that the entire IR field still uses today.
- **1972 - Inverse Document Frequency (Spärck Jones):** Karen Spärck Jones proposed weighting terms by how rarely they appear across documents. This single idea remains the foundation of every term-weighting scheme in use, including BM25.
- **1976 - Probabilistic Retrieval / BIR (Robertson):** Stephen Robertson formalized retrieval as a probability estimation problem: how likely is this document to be relevant? This gave term weighting a theoretical grounding beyond heuristics.
- **1980 - Porter Stemmer (Porter):** Martin Porter published a simple suffix-stripping algorithm that reduced English words to common stems. Despite its errors, it became the de facto standard and is still the default stemmer in many search engines.
- **1994 - BM25 (Robertson et al.):** The Okapi BM25 ranking function combined saturating term frequency, document-length normalization, and probabilistic IDF into a single formula. It outperformed all prior models and remains the default in production search systems.
- **1999 - Apache Lucene (Cutting):** Doug Cutting released Lucene as open-source Java, making high-quality full-text search accessible to any developer. It became the engine underneath Solr, Elasticsearch, and OpenSearch.

- **2010 - Elasticsearch era:** Elasticsearch wrapped Lucene in a distributed, JSON-based API. Suddenly, BM25-powered search scaled to billions of documents with minimal configuration.
- **2020s - BM25 in RAG pipelines:** Large language models need retrieved context to answer questions accurately. BM25 serves as the fast first-stage retriever in Retrieval-Augmented Generation, proving that 30-year-old algorithms still earn their place in modern AI systems.

What you'll learn

- Explain how retrieval systems evolved from Boolean filtering to ranked retrieval (retriever-only arrow.r retriever-ranker)
- Describe the feature extraction pipeline: extract, split, tokenize, and summarize
- Compare document representations: set-of-words vs. bag-of-words vs. tf-idf weighted vectors
- Implement and contrast retrieval models: Boolean, Extended Boolean, Vector Space, Probabilistic (BIR), and BM25
- Evaluate term weighting using inverse document frequency and Zipf's law

Prerequisites

- Basic set theory and linear algebra (dot product, vector norms)
- Familiarity with logarithms and probability notation
- The concept of the semantic gap introduced in the opening chapter

Retrieval Systems and Flows

The need to search large text collections has its roots in library science. As described in the opening chapter, librarians and researchers relied on card catalogs, subject indexes, and classification systems for centuries. When collections outgrew manual methods, the first computerized retrieval systems automated what librarians had always done: match a user's query against a catalog of documents. Today, the same principles apply far beyond libraries, from legal databases and patent archives to corporate knowledge bases and web search engines.

Across these applications, a retrieval system must answer three connected questions: what unit should be returned, what work can be completed before a query arrives, and whether matching documents should be filtered or ranked. These decisions determine the system architecture and the flow from source documents to results.

From Filtering to Ranking

Consider a university library with millions of articles and books. A researcher enters terms like "neural plasticity AND memory consolidation". The simplest architecture retrieves all documents whose catalog entry satisfies this Boolean expression and returns the matching set. This retriever-only approach can evaluate each document independently and return results as soon as it finds them. No scoring or sorting is required. Figure 1.1 illustrates this basic architecture.

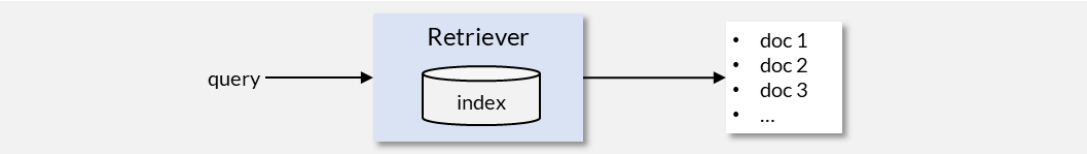


Figure 1.1: Retriever-only architecture: query enters, matching documents come out.

As the library's collection grows, a Boolean query like "neural plasticity" might return hundreds of articles. The researcher needs a way to narrow results without reformulating the query. A post-processing step adds **faceted search**: the ability to filter and sort results by metadata such as publication year, journal, or language. The researcher can toggle these filters while browsing without resubmitting the original search. Faceted search does not change which documents are considered relevant. It simply helps navigate large result sets. Figure 1.2 shows this extended pipeline.

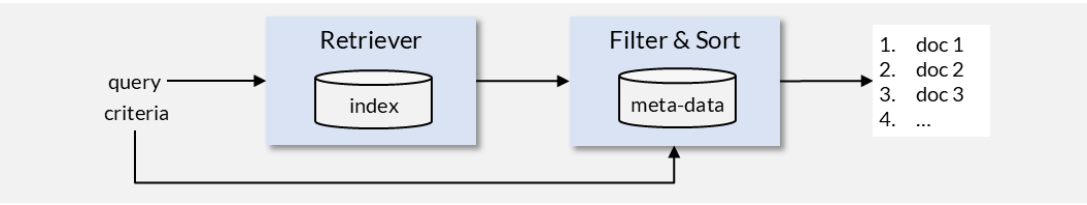


Figure 1.2: Retriever with faceted filtering: candidates are filtered and sorted by metadata.

Even with filters, an unranked list of 200 articles forces the researcher to scan them manually. In the 1970s, the Vector Space and Probabilistic retrieval models introduced a fundamentally different idea: instead of a binary relevant/not-relevant decision, the system estimates a degree of relevance and produces a ranked list. The most promising articles appear first. The architecture becomes a two-stage pipeline: a fast retriever selects candidates, and a ranker scores and orders them (Figure 1.3).

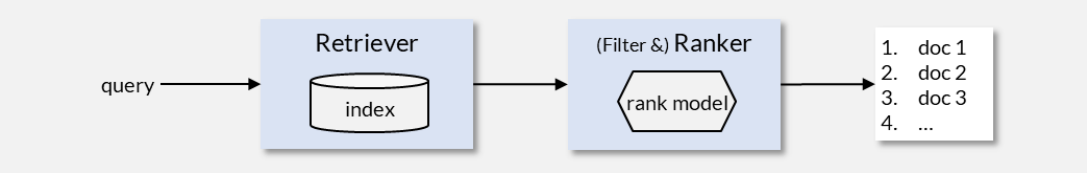


Figure 1.3: Retriever-ranker pipeline: a scoring model orders candidates by estimated relevance.

These three architectural patterns, retriever-only, retriever with faceted filtering, and retriever-ranker, represent the historical progression of text retrieval. The same patterns appear today in systems of all scales, from a researcher querying PubMed to a customer searching an e-commerce catalog.

Documents and Retrieval Granularity

Before we can search a collection, we need to define what a "document" is. In retrieval, a document is the unit that gets indexed, matched against queries, and returned to the user. It consists of three elements:

- **Document ID** - a unique identifier (ideally a UUID) for efficient storage and lookup.
- **Metadata attributes** - descriptive fields (author, publication date, language) used for filtering and faceted search, but not necessarily included in full-text matching.
- **Content attribute(s)** - the text fields indexed for search and ranking.

The critical design decision is: what constitutes a single retrieval unit?

Consider a pharmacology reference book with 800 pages. If we treat the entire book as one document, a search for "ibuprofen dosage" returns "the pharmacology textbook" and the user must search within the book manually. If instead we treat each section or page as a separate document, the system can return "page 47: Ibuprofen - Dosage and Administration" directly.

This decision about retrieval granularity defines the collection structure. Finer granularity gives more specific results but increases the number of entries the system must manage. Coarser granularity reduces index size but forces users to locate relevant passages themselves.

For classical text retrieval, the choice is typically straightforward: emails are individual documents, web pages are individual documents, book chapters or sections become individual documents. The retrieval models in this chapter assume that this splitting has already occurred and each document in the collection is a coherent, self-contained unit. We describe the concrete splitting strategies in the feature extraction pipeline.

> **Note**
>
> Choosing the right retrieval granularity becomes significantly more complex when building systems that feed passages to language models. We revisit chunking strategies, overlapping windows, and hierarchical approaches in the chapter on Retrieval-Augmented Generation.

Offline Phase: Indexing Pipeline

Many search systems scan through data at query time. File search on a local computer, for example, reads through every file on disk whenever the user types a query. This works for a personal laptop with a few thousand files but not for a library with millions of articles. Instead, retrieval is split into two phases: an offline indexing phase that processes documents before any query arrives, and an online querying phase that handles user requests in real time.

The offline phase processes documents through a sequence of stages before any query arrives (Figure 1.4): (a) ingest raw documents from their source formats, (b) split them into retrieval units at the chosen granularity, (c) identify metadata attributes for filtering, (d) extract and normalize text into feature vectors, and (e) store these vectors in a searchable index.

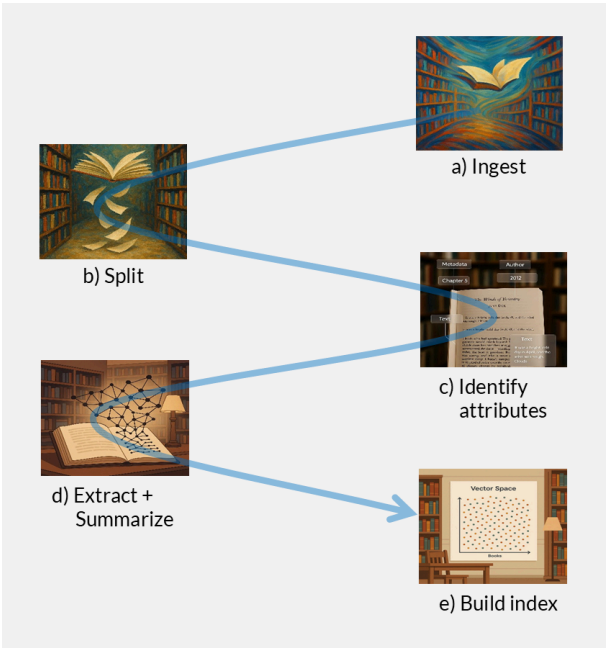


Figure 1.4: Document preprocessing pipeline: ingest (a), split (b), identify attributes (c), extract features (d), build index (e).

Online Phase: Query Processing

When a researcher types "neural plasticity AND memory consolidation" into the library search, the online phase begins. The system processes the query through a symmetric sequence of stages (Figure 1.5): (a) accept the user's query, (b) tokenize it using the same pipeline that processed documents during indexing, (c) optionally broaden the query through spelling correction or synonym expansion, (d) compare the query's feature vector against the vectors stored in the index to find similar documents, and (e) rank candidates by estimated relevance and return an ordered result list.

This symmetry between offline and online processing is essential: queries and documents must live in the same feature space for comparison to work.

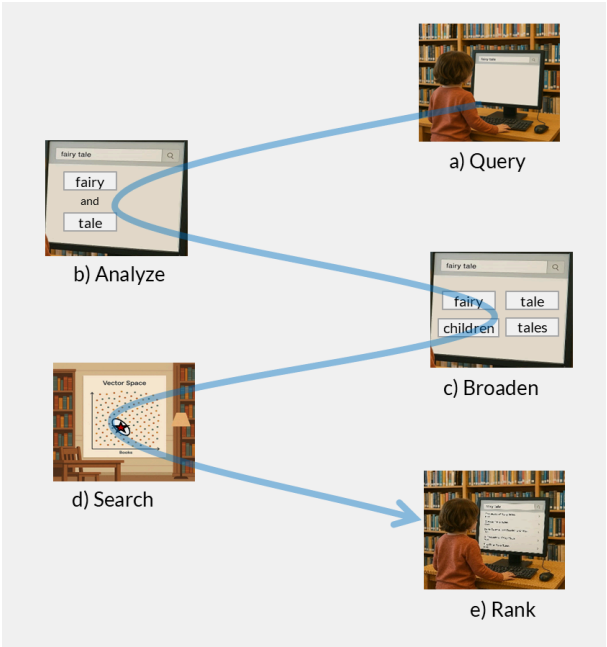


Figure 1.5: Online query pipeline: query (a), tokenize (b), expand (c), search (d), rank (e).

The central challenge in the online phase is relevance ranking: accurately estimating how important a document is given the query. The next sections develop the feature extraction pipeline that creates document representations, followed by the retrieval models that use those representations for scoring.

Hands-on: Explore a Document Collection

Load a document collection, inspect its structure, and understand the raw material that retrieval systems work with.

Open notebook ->

From Text to Searchable Vectors

Retrieval models cannot operate directly on PDF files, HTML markup, or character streams. They compare structured representations of documents and queries. The feature extraction pipeline shown in Figure 1.6 creates these representations through five conceptual steps: extract text from the source format, split it into retrieval units, tokenize and normalize the text, construct a vocabulary, and summarize each unit as a feature vector. This produces two key artifacts:

- **Index** - high-dimensional feature vectors for every retrieval unit, stored together with source metadata.
- **Vocabulary** - the complete set of normalized terms used for both document and query processing.

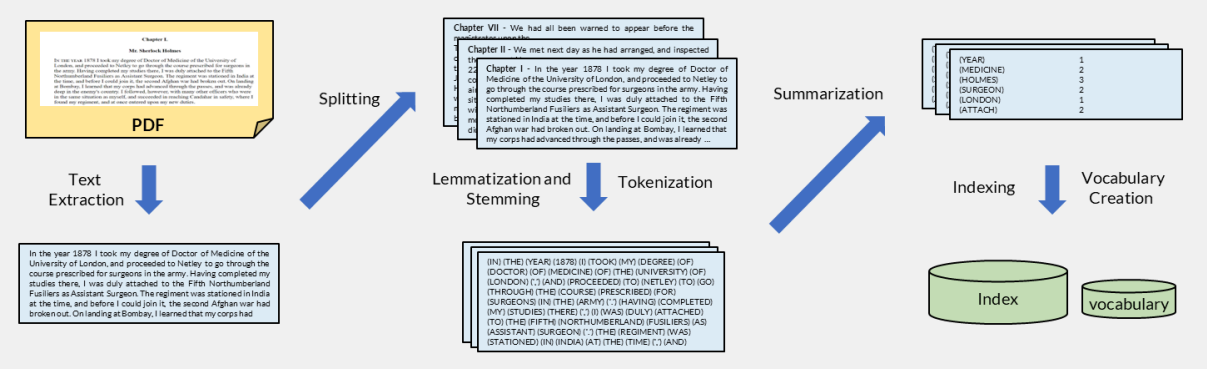


Figure 1.6: Text indexing pipeline: extract, split, tokenize, stem, summarize, then build the index and vocabulary.

Text Extraction

The first step extracts plain text and metadata from the source format. Documents arrive in many formats: PDF, EPUB, plain text, HTML, or office documents (DOCX, PPTX). Each format encodes content differently, mixing actual text with layout instructions, styling, and structural markup. The extraction step strips formatting and control sequences, isolates the character stream, and records metadata attributes (author, title, date) for later use in filtering.

Consider the HTML example in Figure 1.7. The header holds structured metadata (title, keywords) that can enrich the index entry, while the body contains the main text interleaved with markup tags. Extracting useful text from HTML requires parsing the DOM tree and distinguishing content nodes from navigation, scripts, and boilerplate. Although HTML follows a well-defined standard, real-world pages use diverse layouts, making robust extraction (often called scraping) a non-trivial engineering problem.

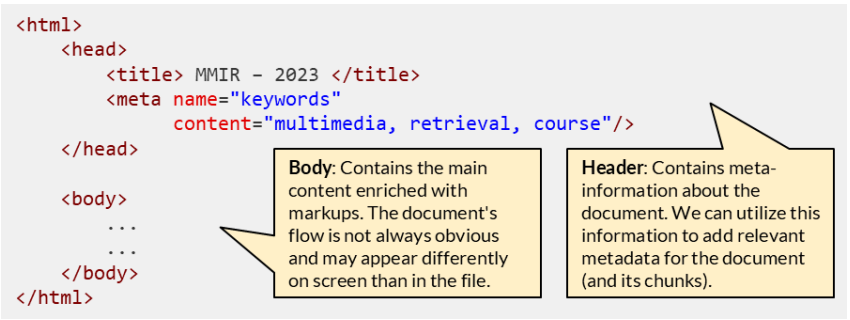


Figure 1.7: HTML document structure: metadata in the header, content in the body.

Source formats differ, so extraction normally relies on format-specific parsers and established libraries. The retrieval pipeline itself only requires that this stage return clean text and useful metadata; the implementation details can be selected for the collection at hand.

Source formats and extraction tools (optional reading)

Different source formats require different extraction strategies:

- **PDF**: Text is stored as positioned glyphs, not logical paragraphs. Extraction must reconstruct reading order from coordinates. Tables and multi-column layouts are particularly challenging.
- **Office documents (DOCX, PPTX)**: ZIP archives containing XML. Libraries can parse the XML structure to extract text, but embedded images and charts require separate handling.
- **HTML/XML**: Well-structured but noisy. Useful text must be separated from navigation menus, advertisements, and script blocks.
- **Plain text and Markdown**: Minimal transformation needed, but encoding detection (UTF-8 vs. legacy encodings) and line-break conventions still require attention.

In practice, we rarely build extraction from scratch. Modern extraction pipelines rely on established libraries and frameworks:

- **Apache Tika**: Java-based toolkit from the Apache Lucene ecosystem that auto-detects file types and extracts text and metadata from over 1,000 formats (PDF, DOCX, EPUB, email archives, images via OCR). Commonly used as the ingestion layer in Solr and Elasticsearch pipelines.
- **Unstructured** (`unstructured` on PyPI): Open-source Python library designed for LLM/RAG pipelines. Handles PDFs, HTML, images, and office documents with built-in chunking support.
- **MarkItDown** (`markitdown` on PyPI): Open-source Python library from Microsoft that converts PDF, DOCX, PPTX, XLSX, HTML, CSV, JSON, images, and audio into clean Markdown. Lightweight and popular for RAG preprocessing.
- **Trafilatura**: Focused on web content extraction. Strips boilerplate from HTML pages and returns clean article text.
- **PyMuPDF (fitz)**: Fast PDF text and layout extraction in Python, preserving reading order and table structure.
- **BeautifulSoup / lxml**: HTML/XML parsers for custom extraction logic when off-the-shelf tools fall short.

The choice of extraction tool depends on the document mix in the collection, the required fidelity (do you need tables? images? footnotes?), and whether the pipeline must run at scale.

At this point, we must also decide on a character encoding for the index. UTF-8 is the dominant standard and handles virtually all scripts. Legacy collections may contain documents in older encodings (Latin-1, Shift-JIS) that require detection and conversion before indexing.

Web documents and link structure

HTML documents carry additional structure beyond text: anchor texts, heading hierarchies, and hyperlink networks. These provide valuable signals for retrieval but require specialized treatment. We cover HTML-specific extraction and link-based relevance (including PageRank) in the chapter on Web Retrieval.

Splitting

As discussed in the previous section, retrieval granularity determines what constitutes a single retrieval unit. Once we have extracted the raw text from a document, we must split it into the units that will be indexed and returned to the user. In classical text retrieval, the splitting strategies are pragmatic and structure-driven:

- **By document boundary**: Each file, email, or web page becomes one retrieval unit. This is the most common approach when documents are naturally self-contained.
- **By structural element**: Books are split at chapter or section headings. Legal texts are split by article or paragraph number. The document's own structure defines the boundaries.
- **By line or log entry**: In observability systems like Elasticsearch, each log line becomes an independent retrieval unit. This enables searching millions of events by timestamp, severity, or content.
- **By fixed page or paragraph count**: When no structural markup is available (e.g., scanned documents), we split at regular intervals such as every N paragraphs or every page.

These approaches share a common property: the splitting decision is made once during indexing and remains fixed. Each resulting unit is treated as an independent document by the retrieval models that follow. Figure 1.8 illustrates this for a novel split into chapter-sized retrieval units.

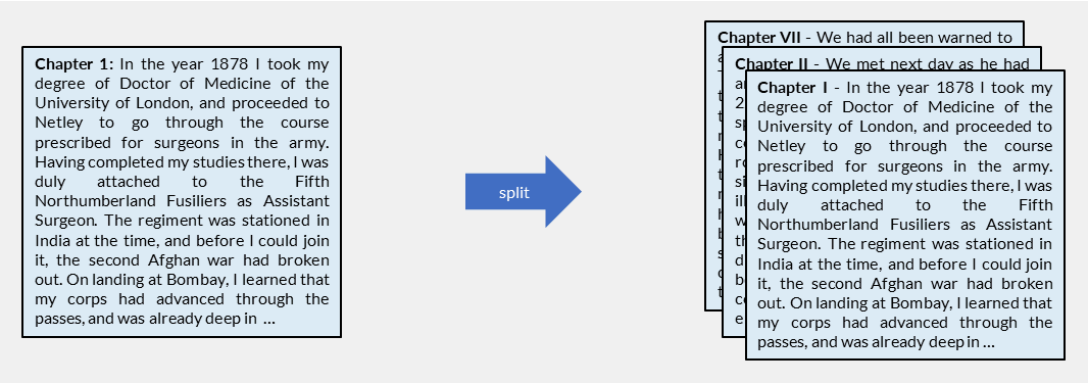


Figure 1.8: A single document divided into multiple smaller retrieval units.

Advanced chunking strategies for RAG

When building systems that feed passages to language models, splitting becomes significantly more nuanced. Overlapping windows, semantic chunking, and hierarchical approaches are covered in the chapter on Semantic Search.

Tokenization

Tokenization converts the extracted character stream into a sequence of discrete units called tokens. A **token** is an individual element in this sequence: it can be a word, a number, or a punctuation mark. A **term** is the normalized form of a token after processing (stemming, case folding). In classical text retrieval, tokens are typically whole words, and the two concepts are often used interchangeably. In later chapters, we will see that this equivalence breaks down when tokens become subword fragments or multi-word phrases.

For classical retrieval, we split text at whitespace and punctuation boundaries to produce word-level tokens. This requires decisions about edge cases: how to handle hyphenated words (“state-of-the-art”), numbers (“3.14”), abbreviations (“U.S.A.”), and possessives (“Watson’s”). In languages without explicit word boundaries (e.g., Japanese, Chinese), segmentation requires dictionary-based or statistical methods.

The most significant challenge is variation in word forms. Since retrieval models match documents to queries by comparing tokens, two tokens are either identical or unrelated. There is no notion of “almost the same”. A query for “cats” will not match a document containing only “cat” because these are distinct tokens. Similarly, “go”, “goes”, “went”, and “going” are four independent dimensions in the vocabulary despite sharing a meaning. The next subsection addresses this through stemming and lemmatization, which reduce inflected forms to a common token before indexing. Figure 1.9 shows the tokenization step applied to a document collection.

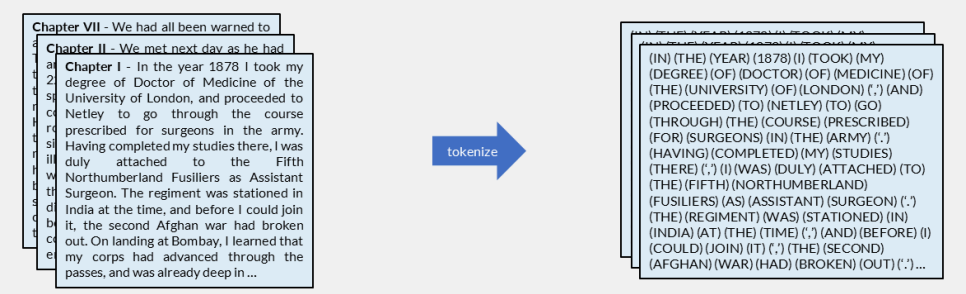


Figure 1.9: Tokenization: raw text split into individual word-level tokens.

Beyond word-level tokens

Subword tokenization (BPE, WordPiece) and n-gram approaches are covered in the chapter on Advanced Text Processing. Word and sentence embeddings, which map tokens to dense vectors, are introduced in the chapter on Semantic Search.

Lemmatization and Stemming

The previous subsection established that retrieval models treat each token as an independent symbol: "cat" and "cats" are unrelated unless we normalize them to the same form. The goal of this pipeline stage is to reduce surface variation so that semantically equivalent word forms map to a single canonical token in the vocabulary.

Two approaches exist:

- **Stemming** applies rule-based suffix stripping to produce a common pseudo-stem. The stem is not necessarily a real word (e.g., "computing", "computation", "computer" all reduce to "comput"). Stemming is fast, requires no dictionary, and works well enough for most retrieval tasks. Its errors go in both directions: it can merge unrelated words ("universal" and "university" both stem to "univers") or fail to merge related ones ("alumnus" and "alumni" remain distinct).
- **Lemmatization** uses a dictionary or morphological analysis to map each word to its actual root form (the lemma). "Better" maps to "good", "went" maps to "go". This produces more accurate results but requires language-specific resources and is computationally more expensive. Modern NLP libraries like spaCy provide lemmatization out of the box.

For classical text retrieval, stemming is the standard choice due to its speed and simplicity. The most widely used stemmer for English is the Porter Algorithm, which we examine next.

Beyond inflection: compounds, synonyms, and semantic relations

Some languages form compound words of arbitrary length (e.g., German, Finnish), creating additional matching challenges. Compound decomposition, synonym expansion, and other linguistic transformations are covered in the chapter on Advanced Text Processing.

The Porter Algorithm

The most well-known rule-based stemmer for English is the Porter Algorithm (1980); see **An Algorithm for Suffix Stripping** in the Further Reading section of the chapter summary. It applies an ordered sequence of suffix-stripping rules to produce a shared pseudo-stem. For the retrieval pipeline, the important result is that related forms map to the same vocabulary term; the exact rule sequence is secondary.

Consider these examples:

Input	Porter stem	True root
"computing", "computation", "computer"	comput	compute
"retrieval", "retrieving", "retrieved"	retriev	retrieve
"generalization", "generalizing"	general	generalize
"relational"	relat	relate
"agreed", "agreeable"	agre	agree

The stems "comput", "retriev", and "agre" are not dictionary words. They exist only to ensure that related forms map to the same token during indexing. When a user queries "computation", the system stems it to "comput" and matches documents containing "computing" or "computer" because those were also stemmed to "comput".

This approach is fast (roughly 60 rules in about 400 lines of code), requires no dictionary, and works well enough for most English retrieval tasks. However, it is purely mechanical and makes errors in both directions: over-stemming merges words that should remain distinct ("operate" and "operating" correctly

merge, but “operation” and “operational” may conflate with “opera”), while under-stemming fails to merge irregular forms (“be”, “was”, “been” remain three separate tokens).

> **Note**
>
> The internal mechanics of Porter are covered in detail in \#advanced-text-processing, along with modern alternatives like Snowball and dictionary-based lemmatization.

Vocabulary

After tokenization and stemming, we have a set of normalized tokens across the entire collection. The **vocabulary** is the complete set of distinct terms that appear at least once. Each term becomes a dimension in the feature space used for retrieval. But not all terms are equally useful.

Stop Words

Many terms are grammatically necessary but carry no content. The article “the” appears in virtually every English document but tells us nothing about what a document is about. A search for “the” would return the entire collection, unable to differentiate between relevant and non-relevant results. Figure 1.10 shows that the 50 most frequent terms in a news corpus account for roughly one-third of all occurrences and appear in over 60% of documents.

t_j	$df(t_j)$	$tf(D,t_j)$
the	100%	6.03%
a	99%	2.57%
of	99%	2.55%
and	99%	2.40%
to	99%	2.58%
in	99%	2.00%
for	98%	0.97%
that	96%	1.15%
on	96%	0.75%
is	95%	0.93%
with	95%	0.70%
at	93%	0.55%
by	92%	0.49%
it	92%	0.69%
as	91%	0.57%
but	91%	0.49%

from	90%	0.44%
be	88%	0.46%
an	88%	0.38%
have	88%	0.45%
was	85%	0.65%
not	84%	0.38%
this	83%	0.35%
are	83%	0.45%
has	83%	0.40%
who	81%	0.37%
they	78%	0.37%
he	78%	0.70%
one	77%	0.26%
said	77%	0.70%
more	75%	0.26%
about	75%	0.27%
or	75%	0.31%

when	74%	0.24%
their	71%	0.27%
his	70%	0.49%
had	70%	0.29%
been	70%	0.21%
all	69%	0.20%
which	69%	0.20%
will	68%	0.27%
out	68%	0.20%
up	68%	0.20%
if	67%	0.21%
than	66%	0.18%
were	66%	0.22%
would	65%	0.23%
can	65%	0.20%
new	64%	0.23%
there	64%	0.18%

Figure 1.10: Document frequency and collection term frequency for the 50 most frequent terms in a news corpus (∼20,000 documents).

Stop word lists for most languages are readily available, for example on Kaggle (stop words in 28 languages). However, eliminating stop words entirely creates problems. Consider the search for “it”: if we remove this term, we lose the ability to find IT books or the novel “It” by Stephen King. The modern approach retains all terms but assigns them different weights based on their discriminating power.

Zipf’s Law

The distribution of term frequencies follows a remarkably regular pattern. Let N be the total number of token occurrences in the collection and M the number of distinct terms. If we rank terms by decreasing frequency, Zipf’s law states that the probability of encountering the term at rank r is:

$$p_r = \frac{c}{r} = \frac{\text{tf}(t)}{N}, \quad \text{term } t \quad \text{with } \text{rank}(t) = r \tag{1.1}$$

The constant c depends only on the vocabulary size:

$$c = \frac{1}{\sum_{r=1}^{M} \frac{1}{r}} \approx \frac{1}{0.5772 + \ln M} \tag{1.2}$$

In other words, a few terms dominate the collection while the vast majority are rare.

For a collection with $M = 5,000$ terms, $c \approx 0.11$; with $M = 100,000$, $c \approx 0.08$. The practical implication is shown in Figure 1.11: terms fall into three zones. The most frequent terms (above the upper cut-off) appear everywhere and cannot discriminate. The rarest terms (below the lower cut-off) are highly specific but unlikely to occur in queries, making them less useful despite their discriminating potential. The terms most useful for distinguishing relevant from non-relevant documents lie in between.

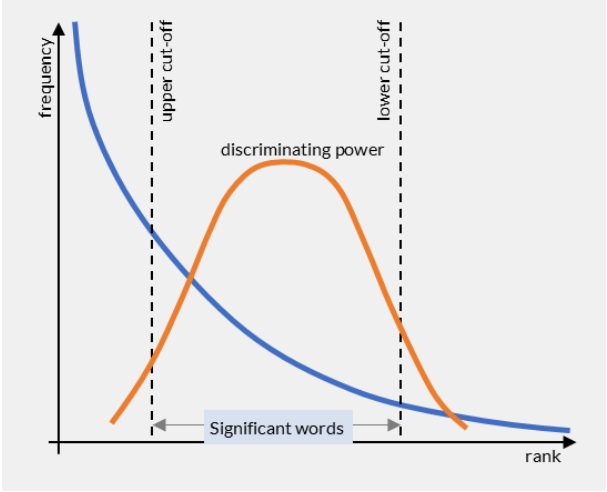


Figure 1.11: Zipf's law and term discrimination: the most useful terms for retrieval fall between the frequency extremes.

Term Discrimination and IDF

The intuition behind term weighting is straightforward: imagine removing a single term from every document in the collection. If documents suddenly become harder to tell apart, that term was helping to discriminate between them. Conversely, removing a term that appears everywhere (like "the") changes nothing. Terms that appear in a moderate number of documents have the highest discriminating power, as confirmed empirically in Figure 1.12.

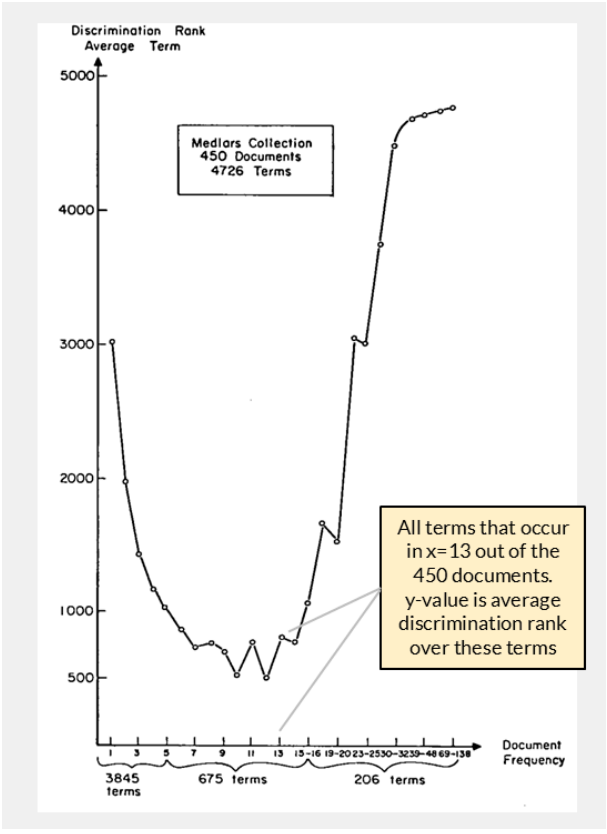


Figure 1.12: Discrimination rank vs. document frequency in the Medlars collection (450 documents). Terms occurring in 9-12 documents have the highest discrimination rank. Source: Salton, Wong & Yang (1975).

Karen Spärck Jones (1972) captured this intuition in a single formula called the **inverse document frequency** (IDF); see **A Statistical Interpretation of Term Specificity and Its Application in Retrieval** in the Further Reading section of the chapter summary. It builds on one statistic: the **document frequency** $\mathrm{df}(t)$, which counts how many documents contain term t at least once. IDF then weights terms inversely to their commonness:

Key Formula: Inverse Document Frequency

$$\text{idf}(t) = \log \frac{N+1}{\text{df}(t)+1} \tag{1.3}$$

Terms that appear in fewer documents receive higher IDF weights. The "+1" in both numerator and denominator is a smoothing term that avoids division by zero (for terms not yet seen) and ensures that a term appearing in every document still receives a small positive weight rather than exactly zero.

Example

Consider a collection of $N = 10,000$ documents. We compute IDF for four terms:

Term	$\text{df}(t)$	$\text{idf}(t) = \log \frac{10,001}{\text{df}+1}$	Interpretation
"the"	9,800	$\log \frac{10,001}{9,801} \approx 0.009$	Appears almost everywhere, nearly zero weight
"retrieval"	50	$\log \frac{10,001}{51} \approx 2.29$	Moderately rare, high weight
"okapi"	3	$\log \frac{10,001}{4} \approx 3.40$	Very rare, highest weight
"algorithm"	1,200	$\log \frac{10,001}{1,201} \approx 0.92$	Common but not ubiquitous, moderate weight

A query "retrieval algorithm" would weight "retrieval" roughly 2.5× more than "algorithm" because it appears in far fewer documents. The stop word "the" contributes almost nothing even if present in the query.

IDF provides the weighting mechanism we need: common terms get low weights, rare terms get high weights. Combined with term frequency, it produces the **tf·idf** weighting that we will use for some of the retrieval models in the next sections. Figure 1.13 compares IDF weights with empirical discrimination power, confirming that IDF closely approximates the true discriminating value of terms.

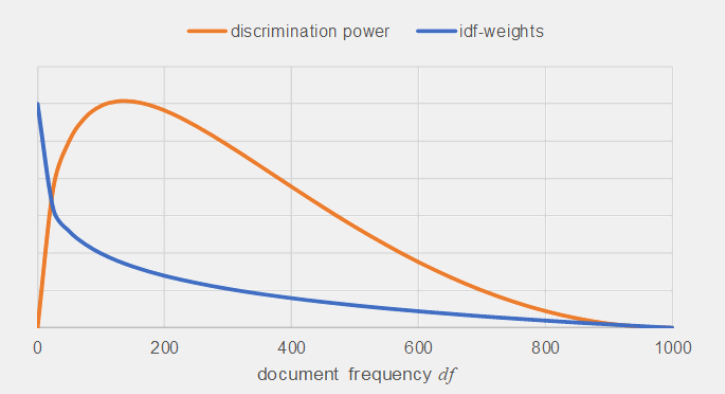


Figure 1.13: IDF weights closely track empirical discrimination power as a function of document frequency (N = 1,000).

Document Representation

With the vocabulary established and each term assigned an IDF weight, we can now represent documents as vectors. Each document becomes a point in an M-dimensional space where M is the vocabulary size and each dimension corresponds to one term. Figure 1.14 shows this process: tokenized text is stemmed, stop words are removed, and the remaining terms are counted to produce per-document frequency vectors.

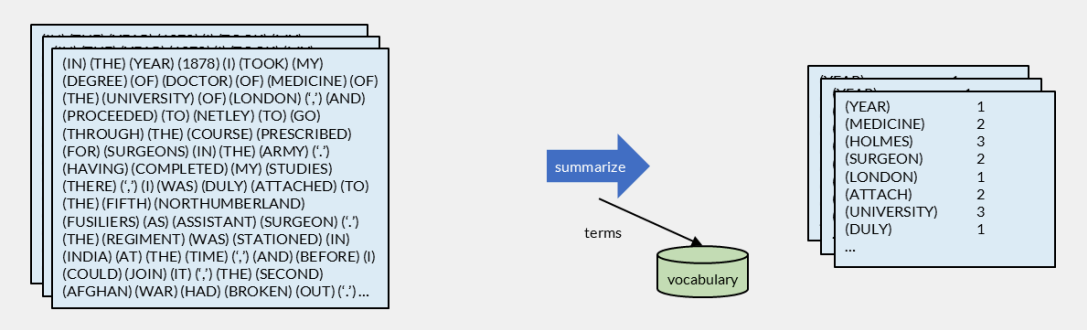


Figure 1.14: From tokenized text to term-frequency vectors: stem, remove stop words, count occurrences per document.

Set-of-Words and Bag-of-Words

Let D_i be the i-th document in a collection of N documents. The feature representation $\boldsymbol{d}_i \in \mathbb{R}^M$ is a vector where component $d_{i,j}$ describes how term t_j occurs in document D_i. We use $\mathrm{tf}(D_i, t_j)$ to denote the number of occurrences of term t_j in document D_i.

The simplest option is the **set-of-words** model: for each term, record only whether it is present (1) or absent (0). A document mentioning "cat" three times looks identical to one mentioning it once:

$$d_{i,j} = \tag{1.4}$$

A richer option is the **bag-of-words** model: record how many times each term appears. A document mentioning "cat" five times is represented differently from one mentioning it once:

$$d_{i,j} = \mathrm{tf}(D_i, t_j) \tag{1.5}$$

Both models ignore term proximity and order. A document containing "New York" is represented the same way as one containing "York New". Despite this limitation, these representations work remarkably well because the presence and frequency of specific terms are strong signals of relevance.

TF·IDF Weighting

The bag-of-words model treats all terms equally: a document with 10 occurrences of "the" scores the same as one with 10 occurrences of "algorithm". By combining term frequency with IDF, we produce weighted vectors where informative terms contribute more:

Key Formula: tf·idf document representation

$$d_{i,j} = \mathrm{tf}(D_i, t_j) \cdot \mathrm{idf}(t_j) \tag{1.6}$$

This **tf·idf** weighting is the standard document representation for classical retrieval models.

Sparsity

In practice, a vocabulary can include millions of terms. However, most documents contain only a few hundred or thousand unique terms. The feature vectors are therefore sparsely populated: the vast majority of components are zero. Efficient storage methods like the inverted file exploit this sparsity by recording only the non-zero entries. During retrieval, the system only needs to consider documents that share at least one term with the query.

Hands-on: Feature Extraction

Build a feature extraction pipeline in Python: tokenize text, apply stemming, compute term frequencies and IDF weights.

Open notebook ->

Boolean Retrieval

The previous section transformed raw text into tokens and term-frequency vectors. A retrieval model takes these representations and decides which documents match a query and, for ranked models, how strongly each document matches. We use the following small fictional library catalogue as a running example. Its wording is deliberately controlled so that the effects of preprocessing, term weighting, document length, and lexical ambiguity remain visible.

ID	Catalogue entry
D_1	**The Cat and Dog in the Forest.** A cat and a dog begin a woodland adventure.
D_2	**Cats of the Woodland.** Wild cats begin an adventure beneath ancient trees.
D_3	**The Forest Hound.** A loyal dog follows a woodland trail through ancient trees.
D_4	**Feline Detective.** A clever cat solves mysteries with a canine companion.
D_5	**Random Forest for Pet Detection.** A random forest model classifies cats and dogs in photographs.
D_6	**Forests of Search Trees.** Algorithms explore a forest of binary trees and graph paths.
D_7	**Dog Dog Dog!** A dog chases a ball, finds a bone, and wakes the neighbors.
D_8	**Forest Forest Forest!** A forest guide names trees, flowers, rivers, birds, and hidden ruins.
D_9	**Cat Dog Forest.**
D_{10}	**Cat Dog Forest.** A cat meets a dog in a forest beside rivers, mountains, castles, villages, bridges, and caves.
D_{11}	**Woodland Companions.** A kitten and a puppy share a moonlit adventure among old trees.
D_{12}	**The Pet Bakery.** A cat, a dog, and a baker make cakes, bread, biscuits, pies, and coffee.

Following From Text to Searchable Vectors, we must define the preprocessing pipeline before applying a retrieval model. For this example, we lowercase the text, split it at punctuation and whitespace, and remove common stop words. We deliberately apply neither stemming nor lemmatization, and we do not expand synonyms. For example:

$D_1 \mapsto [\text{cat}, \text{dog}, \text{forest}, \text{cat}, \text{dog}, \text{begin}, \text{woodland}, \text{adventure}]$

$D_2 \mapsto [\text{cats}, \text{woodland}, \text{wild}, \text{cats}, \text{begin}, \text{adventure}, \text{beneath}, \text{ancient}, \text{trees}]$

These choices expose an important property of lexical matching: terms match by identity, not by visual or semantic similarity. During vocabulary construction, each distinct token receives its own identifier. For example, the vocabulary might map “cat” to token ID 70 and “cats” to token ID 83. A retrieval model therefore treats them as two unrelated dimensions, even though a reader immediately recognizes their relationship. For the same reason, “cat” does not match “feline” or “kitten”, and “dog” does not match “hound”, “canine”, or “puppy”. The reverse problem occurs with “forest”: the same token ID represents both a natural environment and the technical concept in “random forest”, although the meanings differ.

Standard Boolean Model

Consider the query `cat AND dog`. For each document, we test two conditions: the token “cat” is present, and the token “dog” is present. The complete predicate evaluates to true only if both conditions are met. The result set is therefore $\{D_1, D_9, D_{10}, D_{12}\}$. For `cat OR dog`, the predicate evaluates to true if at least one condition is met, producing the larger result set $\{D_1, D_3, D_4, D_7, D_9, D_{10}, D_{12}\}$.

This is the central idea of the Standard Boolean Model: a query is a predicate that maps each document to either true or false. Documents for which the predicate is true enter the result set; all others are excluded. The model filters rather than ranks, so every returned document has the same status regardless of how often or where the terms occur. This makes Boolean retrieval simple to understand, easy to calculate, and straightforward to explain.

Query Language

The query grammar defines two atomic predicates: a term is present, or a term is absent. It then defines two composition rules: AND and OR combine existing predicates into more complex expressions. Formally:

$$Q = t \quad (\text{term } t \text{ must be present}) \tag{1.7}$$

$$Q = \neg t \quad (\text{term } t \text{ must be absent}) \tag{1.8}$$

$$Q = Q_1 \vee Q_2 \quad (\text{at least one sub -query must be true}) \tag{1.9}$$

$$Q = Q_1 \wedge Q_2 \quad (\text{both sub -queries must be true}) \tag{1.10}$$

For example, $(\text{cat} \wedge \text{dog}) \vee \text{hound}$ returns a document if it contains both "cat" and "dog", or if it contains "hound". Boolean algebra allows any such expression to be rewritten in disjunctive normal form (DNF):

Key Formula: Boolean Query in DNF

$$Q = \bigvee_{l=1}^{L} \left(\bigwedge_{k=1}^{K_l} \tau_{l,k} \right) \tag{1.11}$$

with $\tau_{l,k} = t_{j(l,k)}$ or $\tau_{l,k} = \neg t_{j(l,k)}$, where $j(l,k)$ maps to the index of the term used in the query.

Any Boolean query can be rewritten as a disjunction (OR) of conjunctions (AND). This normal form enables systematic evaluation using set operations.

Evaluating a Query

The direct evaluation method follows the initial intuition. For each document D_i, the system checks whether every atomic predicate $\tau_{l,k}$ is true or false and then evaluates the complete expression. For `cat AND dog`, D_1 produces $\text{true} \wedge \text{true} = \text{true}$, whereas D_4 produces $\text{true} \wedge \text{false} = \text{false}$.

A second method starts from all documents that satisfy each atomic predicate. Let $\mathbb{S}_t$ denote the set of documents containing term t. For the running collection:

$\mathbb{S}_{\text{cat}} = \{D_1, D_4, D_9, D_{10}, D_{12}\}$

$\mathbb{S}_{\text{dog}} = \{D_1, D_3, D_7, D_9, D_{10}, D_{12}\}$

The documents satisfying `cat AND dog` must belong to both sets. The documents satisfying `cat OR dog` may belong to either set:

$\mathbb{S}_{\text{cat}} \cap \mathbb{S}_{\text{dog}} = \{D_1, D_9, D_{10}, D_{12}\}$

$\mathbb{S}_{\text{cat}} \cup \mathbb{S}_{\text{dog}} = \{D_1, D_3, D_4, D_7, D_9, D_{10}, D_{12}\}$

More generally, each atomic predicate defines a set of matching documents:

$\mathbb{S}_{l,k} =$

The DNF structure translates directly into intersections for AND and unions for OR:

$\mathbb{Q} = \bigcup_{l=1}^{L} \bigcap_{k=1}^{K_l} \mathbb{S}_{l,k}$

An inverted file stores these term-document sets so that the system can combine them without scanning every document. Chapter 4 develops this index structure and its efficient query-processing algorithms.

Limitations of Boolean Retrieval

Suppose the information need is a story involving a cat, a dog, and trees. The strict query `cat AND dog AND trees` returns no documents. Several entries are plausible partial matches, but none contains all three exact tokens. For example, D_3 contains "dog" and "trees", while D_1 contains "cat", "dog", "forest", and "woodland". Boolean evaluation excludes both because a missing term makes the complete AND predicate false.

Relaxing the query to `cat OR dog OR trees` creates the opposite problem: it returns 11 of the 12 documents. The result includes technical entries such as D_6 because it contains "trees", even though these are binary

search trees rather than trees in an animal story. Small changes to a Boolean expression can therefore produce abrupt changes in result-set size, from no results to almost the entire collection.

The model also provides no basis for ordering the matches. For `cat AND dog`, D_1, D_9, D_{10}, and D_{12} are equivalent Boolean matches. The model ignores that D_1 and D_{10} repeat both query terms, that D_9 consists only of the three topical terms "cat", "dog", and "forest", and that much of D_{12} concerns baking. Term frequency, term importance, document length, and partial evidence do not affect the result.

These limitations establish the questions for the models that follow. The Extended Boolean Model introduces graded matching and ranking while retaining Boolean query structure. Probabilistic and vector-space models provide alternative foundations for term weighting and ranking, while BM25 later combines probabilistic term importance with term-frequency saturation and document-length normalization. We will return to the same collection to see which limitation each model addresses.

Advantages: The model has simple and precise query semantics. Its decisions are easy to calculate and explain, and set operations support fast evaluation over large collections. Boolean predicates also combine naturally with structured metadata filters such as `language = English`.

Disadvantages: Users must express their information need as a Boolean expression and may obtain either too few or too many results. The model neither ranks matches nor represents partial matches. All predicates have equal influence, regardless of term frequency or discriminating power. These properties make the model better suited to exact filtering than to ranking documents by relevance.

Boolean retrieval nevertheless remains an important component of modern text and web search systems. A fast first stage often combines query terms with OR to retrieve a broad candidate set. A second stage then assigns relevance scores and ranks these candidates, allowing the user to inspect the strongest matches first. The next model takes an initial step in this direction by replacing Boolean decisions with graded scores.

Extended Boolean Model

The query `cat AND dog` gave D_1, D_9, D_{10}, and D_{12} the same Boolean status. Yet their evidence differs: D_1 and D_{10} contain both terms twice, while D_9 and D_{12} contain them once. Documents such as D_3 and D_4 contain only one query term and are excluded completely. The Extended Boolean Model replaces this hard boundary with a graded score. It can rank exact matches and retain partial matches with lower scores.

What Remains Boolean, and What Changes?

The query language remains the same. Users still express requirements with term predicates, AND, OR, and NOT, and the query retains its tree structure. What changes is the evaluation of that tree. An atomic predicate no longer produces only true or false. It produces a score between 0 and 1 that reflects the strength of the term evidence. Soft versions of AND and OR then combine these scores.

This extension directly addresses two limitations of the Standard Boolean Model. Term frequency and inverse document frequency can distinguish stronger from weaker evidence, while graded operators allow a document to receive a positive score even if it does not satisfy every predicate. The output is therefore a ranked list rather than an unordered set.

Formalizing Graded Evidence

Each document is represented by normalized TF-IDF weights. For term t_j in document D_i, define

$$d_{i,j} = \min\left(1, \frac{\mathrm{tf}(D_i, t_j)\,\mathrm{idf}(t_j)}{\alpha}\right), \qquad 0 \le d_{i,j} \le 1,$$

where α is a fixed normalization value shared by the collection. A positive term predicate uses this weight directly; a negative predicate reverses it:

$$\mathrm{sim}(\tau, D_i) =$$

The remaining question is how to combine the atomic scores. The p-norm model, introduced by Salton, Fox, and Wu; see **Extended Boolean Information Retrieval** in the Further Reading section of the chapter summary, interprets soft AND as closeness to the ideal point $(1, \ldots, 1)$ and soft OR as distance from $(0, \ldots, 0)$.

Key Formula: P-Norm Model

$$\mathrm{sim}\Big(\bigwedge_{k=1}^{K} Q_k, D_i\Big) = 1 - \sqrt[p]{\frac{\sum_{k=1}^{K} (1-\mathrm{sim}(Q_k, D_i))^p}{K}}$$

$$\mathrm{sim}\Big(\bigvee_{k=1}^{K} Q_k, D_i\Big) = \sqrt[p]{\frac{\sum_{k=1}^{K} \mathrm{sim}(Q_k, D_i)^p}{K}}, \qquad 1 \le p < \infty$$

When $p = 1$, both operators average their evidence. As p increases, AND becomes more sensitive to the weakest predicate and OR becomes more sensitive to the strongest predicate. In the limit $p \to \infty$, they approach minimum and maximum.

Other soft Boolean operators are possible. They differ in how strongly one high or low atomic score influences the combined result.

Alternative soft Boolean operators (optional reading)

The fuzzy set model uses the weakest score for AND and the strongest score for OR:

$$\mathrm{sim}(Q_1 \wedge Q_2, D_i) = \min\{\mathrm{sim}(Q_1, D_i), \mathrm{sim}(Q_2, D_i)\}$$

$$\mathrm{sim}(Q_1 \vee Q_2, D_i) = \max\{\mathrm{sim}(Q_1, D_i), \mathrm{sim}(Q_2, D_i)\}$$

This preserves the strict interpretation of AND and OR most directly, but ignores all scores except the extreme one.

The fuzzy algebraic model combines two operands using a product for AND and a probabilistic sum for OR:

$$\mathrm{sim}(Q_1 \wedge Q_2, D_i) = \mathrm{sim}(Q_1, D_i)\,\mathrm{sim}(Q_2, D_i)$$

$$\mathrm{sim}(Q_1 \vee Q_2, D_i) = \mathrm{sim}(Q_1, D_i) + \mathrm{sim}(Q_2, D_i) - \mathrm{sim}(Q_1, D_i)\,\mathrm{sim}(Q_2, D_i)$$

A further family interpolates between the minimum and maximum. For AND, a parameter closer to the minimum emphasizes the weakest condition; for OR, a parameter closer to the maximum emphasizes the strongest condition. These alternatives illustrate that softening Boolean logic requires a modelling choice: Boolean algebra alone does not determine the graded operator.

Evaluating the Running Example

Consider `cat AND dog` with $p = 2$. In the 12-document collection, $\mathrm{df}(\mathrm{cat}) = 5$ and $\mathrm{df}(\mathrm{dog}) = 6$, so

$$\mathrm{idf}(\mathrm{cat}) = \ln(12/5) \approx 0.875, \qquad \mathrm{idf}(\mathrm{dog}) = \ln(12/6) \approx 0.693.$$

For this example, choose $\alpha = 4 \ln 2 \approx 2.773$, the largest TF-IDF weight among these query terms. We compute the two atomic scores for each candidate and combine them with the p-norm AND formula.

Document	$d_{i,\,\mathrm{cat}}$	$d_{i,\,\mathrm{dog}}$	p-norm AND score
D_1	0.632	0.500	0.561
D_{10}	0.632	0.500	0.561
D_7	0.000	1.000	0.293
D_9	0.316	0.250	0.282
D_{12}	0.316	0.250	0.282
D_4	0.316	0.000	0.143
D_3	0.000	0.250	0.116

The model now distinguishes repeated evidence: D_1 and D_{10} rank above D_9 and D_{12}. It also assigns positive scores to the partial matches D_3, D_4, and D_7. Evaluation can still use an inverted file: the union of the query-term postings supplies candidates, after which the system computes and sorts their scores.

Limitations of the Extended Boolean Model

The ranking also exposes the heuristic nature of the method. D_7 contains "dog" four times but no "cat", yet it narrowly outranks D_9, which contains both requested terms. A different value of p, another normalization constant, or another soft operator can change this order. The query grammar states what AND and OR mean for true and false values, but it does not uniquely determine how graded evidence should be combined.

The model also inherits the lexical limitations of its representation. It still does not connect "cat" with "cats" or "feline", and it cannot distinguish the meanings of "forest". Finally, users must still translate an information need into an explicit Boolean expression. Graded evaluation softens the consequences of that expression but does not remove the burden of formulating it.

Advantages: The model retains the familiar Boolean query structure while adding partial matches and ranked output. TF-IDF weights distinguish terms by occurrence and collection frequency, and inverted files still support efficient candidate retrieval.

Disadvantages: Scores depend on heuristic choices for normalization, operators, and parameters, which can produce counter-intuitive rankings. The model retains lexical matching and explicit Boolean query formulation.

The classical p-norm model is rarely used as the primary ranker in modern search engines. Its central idea remains relevant, however: contemporary systems often combine strict Boolean filters with optional scoring clauses. This preserves explicit query constraints while allowing partial matches to receive different relevance scores. The Vector Space Model in the next section takes the next conceptual step: it replaces explicit Boolean expressions with free-text queries and ranks documents directly from their weighted term vectors.

Vector Space Retrieval

The Vector Space Model emerged from the SMART information retrieval project led by Gerard Salton and colleagues in the 1960s, formalized in Salton, Wong, and Yang's **A Vector Space Model for Automatic Indexing**, listed in the Further Reading section of the chapter summary. SMART introduced many ideas that became central to classical text retrieval: weighted terms, document and query vectors, partial matching, and ranked output. The model had a major influence on both research and practical search systems because it replaced exact logical decisions with simple numerical operations.

Consider the free-text query `cat dog forest`. Boolean OR retrieves every document containing at least one of these terms but cannot order the ten matches. The Vector Space Model uses the same tokenized collection and the same term statistics, but represents the query and documents as vectors. Their similarity becomes a score, allowing the system to place stronger matches first.

What Remains the Same, and What Changes?

As before, preprocessing determines the vocabulary, and each distinct token defines one dimension. Documents remain bag-of-words representations, so word order is ignored and lexical variants such as "cat" and "cats" remain separate. IDF still gives less weight to terms that occur in many documents.

The query changes more fundamentally. Instead of a Boolean expression, it is treated as a short document and processed through the same pipeline. No AND or OR operators are required. A document may receive a positive score when it shares only one query term, and the score determines its position in the ranked result list.

Document and Query Vectors

For a vocabulary of M terms, document D_i becomes the sparse vector $\boldsymbol{d}_i = \left(d_{i,1}, \ldots, d_{i,M}\right)$ with

$$d_{i,j} = \mathrm{tf}\left(D_i, t_j\right) \mathrm{idf}\left(t_j\right), \qquad 1 \leq j \leq M.$$

The query becomes a vector in the same space:

$$q_j = \mathrm{tf}\left(Q, t_j\right) \mathrm{idf}\left(t_j\right), \qquad 1 \leq j \leq M.$$

Placing the document vectors as columns produces the term-document matrix $\boldsymbol{A}$. This matrix is conceptually useful, but practical systems store only non-zero entries because each document contains a small fraction of the complete vocabulary.

Inner Product and Cosine Similarity

The inner product, also called the dot product, sums the weighted evidence from terms shared by query and document:

> **Note**
>
> Key Formula: Inner Product

Only query dimensions contribute because $q_j = 0$ for every non-query term. Repeating a query term increases the score linearly. The score has no fixed upper bound and is often called a retrieval status value rather than a similarity probability. For the full collection,

$$\boldsymbol{sim}_{\mathrm{dot}}(Q, \mathbb{D}) = \boldsymbol{A}^{\top} \boldsymbol{q}.$$

Cosine similarity divides the same inner product by both vector lengths:

> **Note**
>
> Key Formula: Cosine Similarity

Cosine similarity compares vector direction rather than magnitude. For non-negative TF-IDF vectors, its value lies between 0 for no shared terms and 1 for identical directions.Document vectors can be normalized

once during indexing. The system then computes cosine similarity as an inner product between unit-length vectors, avoiding repeated norm calculations at query time.

Evaluating a Query

Evaluation begins like Boolean OR. The system looks up the posting list for every query term and takes their union; documents outside this union have score zero. For each candidate, it accumulates $q_j d_{i,j}$ from the matching postings. This sum is already the inner-product score. Cosine evaluation adds one step: divide by the query norm and the document norm stored with the index. Finally, sort candidates by decreasing score.

This procedure avoids constructing the dense matrix $\boldsymbol{A}$ or comparing the query with every document. Chapter 4 develops inverted files and posting-list traversal in detail.

Running Example

For $Q =$ "cat dog forest" and $N = 12$, the query terms have

$$\text{idf}(\text{cat}) = \ln(12/5) \approx 0.875,$$

$$\text{idf}(\text{dog}) = \ln(12/6) \approx 0.693,$$

$$\text{idf}(\text{forest}) = \ln(12/7) \approx 0.539.$$

Each query term occurs once, so these values are also the non-zero components of $\boldsymbol{q}$. Applying the same TF-IDF representation to every document gives the complete ranking evidence below.

Document	TF (`cat`, `dog`, `forest`)	Inner product	Cosine
D_1	(2, 2, 1)	2.784	0.659
D_2	(0, 0, 0)	0.000	0.000
D_3	(0, 1, 1)	0.771	0.102
D_4	(1, 0, 0)	0.766	0.093
D_5	(0, 0, 2)	0.581	0.059
D_6	(0, 0, 1)	0.291	0.034
D_7	(0, 4, 0)	1.922	0.232
D_8	(0, 0, 4)	1.162	0.139
D_9	(1, 1, 1)	1.537	1.000
D_{10}	(2, 2, 2)	3.075	0.342
D_{11}	(0, 0, 0)	0.000	0.000
D_{12}	(1, 1, 0)	1.247	0.137

The inner product ranks $D_{10} > D_1 > D_7 > D_9$. Repeating all query terms makes D_{10} the strongest match, while repeating only “dog” allows D_7 to outrank the compact exact match D_9. Cosine produces a different order: $D_9 > D_1 > D_{10} > D_7$. The vector for D_9 has exactly the query direction and therefore receives the maximum score of 1. The additional non-query terms in D_{10} increase its norm and reduce its cosine score even though it contains every query term twice.

Geometry of inner product and cosine (optional reading)

For a fixed score threshold γ, the inner product accepts documents satisfying $\boldsymbol{q}^\top \boldsymbol{d} \geq \gamma$. The boundary is a hyperplane with the query vector as its normal. Documents farther from the origin in the query direction receive larger scores. Since dimensions with $q_j = 0$ vanish from the sum, the inner product effectively projects each document onto the subspace spanned by the query terms.

Cosine similarity instead defines a hypercone around the query vector. A higher threshold produces a narrower cone, so accepted documents must point in a direction closer to the query. Scaling every

component of a document by the same factor does not change its cosine score. The measure therefore rewards similar proportions rather than high absolute frequencies.

This distinction explains D_9 and D_{10}. Both have query-term proportions $1:1:1$, but cosine is measured in the full vocabulary space. The many non-query components of D_{10} contribute to $|\,\boldsymbol{d}_{10}\,|$ even though they contribute nothing to the numerator. They rotate the full document vector away from the query and lower its score.

For a one-term query, the geometry becomes extreme. The inner product primarily ranks documents by the frequency of that term. Cosine rewards vectors concentrated on that one dimension, so additional vocabulary lowers the score. A page containing little except the query term can therefore outrank a richer document that discusses the same topic in broader language.

Limitations of Vector Space Retrieval

TF-IDF weights and both similarity measures are heuristic. They rank effectively in many collections, but neither formula directly estimates relevance. The inner product grows linearly with term frequency, so long documents tend to receive more query-term occurrences and authors can manipulate rankings by repeating selected terms. In the example, four occurrences of "dog" push D_7 above D_9 despite the absence of "cat" and "forest".

Cosine removes this magnitude bias but introduces different assumptions. It prefers documents whose term-frequency ratios resemble the query and uses every document term in the normalization. Consequently, non-query terms can lower a score, although they provide no negative evidence in the inner-product numerator. This is counter-intuitive when a relevant passage is embedded in a longer document.

The representation also retains the earlier lexical limitations. Terms are independent dimensions, so the model neither connects "cat" with "feline" nor distinguishes the meanings of "forest". Word order and dependencies such as "random forest" are absent from the bag-of-words vector.

Term independence assumption

The Vector Space Model treats terms as independent dimensions. For example, "New" and "York" contribute separately even when their combination denotes one place. Later chapters introduce phrases, latent representations, and dense embeddings that capture some relationships between terms.

Advantages: The model accepts natural free-text queries, supports partial matches, produces ranked output, and gives discriminating terms more influence through IDF. Sparse vectors and inverted files make evaluation simple and efficient.

Disadvantages: Weighting and similarity are heuristic. Inner products favour high term frequencies and longer documents, while cosine can penalize useful additional content and favours query-like term ratios. Neither measure includes term-frequency saturation, robust length normalization, or semantic relationships.

Modern Applications

Classical TF-IDF vectors with cosine similarity remain useful as transparent baselines and in small or specialized collections. They also support document similarity, clustering, classification, recommendation, and near-duplicate detection where a sparse lexical representation is sufficient. For primary lexical ranking, production search systems now commonly prefer BM25 because it controls term-frequency growth and document-length effects more carefully.

The geometry has become even more important than the original representation. Modern semantic search encodes text as dense embedding vectors and retrieves neighbours with inner product or cosine similarity, often through approximate nearest-neighbour indexes. Hybrid systems combine this dense vector retrieval with sparse lexical methods such as BM25. Thus, modern systems frequently retain the Vector Space Model's scoring operations while replacing TF-IDF dimensions with learned semantic features.

The Vector Space Model establishes the geometry of ranked retrieval but leaves term-frequency growth and document-length effects to heuristic choices. The next section introduces probabilistic term evidence and develops BM25 as a practical response to these limitations.

Probabilistic Ranking and BM25

The Vector Space Model ranks effectively, but its weighting and similarity measures are heuristic. Probabilistic retrieval asks a more direct question: given query Q and document D_i, how likely is the document to be relevant? The Binary Independence Model (BIR) develops this idea from binary term evidence. BM25 then retains its probabilistic term weighting while adding the term-frequency saturation and document-length normalization missing from BIR and vector-space scoring.

Binary Independence Model

The BIR model uses the same tokenized vocabulary as the previous models, but represents each query and document as a set of terms. It was formalized by Robertson and Spärck Jones; see **Relevance Weighting of Search Terms** in the Further Reading section of the chapter summary. A component is 1 when a term is present and 0 when it is absent. It assumes that terms contribute independently and that non-query terms occur equally often in relevant and non-relevant documents. Under these assumptions, only query terms present in a document affect its rank.

Let R denote relevance and NR non-relevance for query Q. Ranking by the posterior odds

$$\frac{P(R \texttt{ mid } D_i, Q)}{P(NR \texttt{ mid } D_i, Q)} \tag{1.12}$$

is equivalent to ranking by the document evidence after query-dependent constants are removed. Define

$$r_j = P\big(d_{i,j} = 1 \texttt{ mid } R, Q\big), \qquad n_j = P\big(d_{i,j} = 1 \texttt{ mid } NR, Q\big). \tag{1.13}$$

The first probability measures how often term t_j occurs in relevant documents; the second measures how often it occurs in non-relevant documents. The resulting score is additive, but several probability terms must cancel before we reach that form. The derivation below explains why only query terms present in the document remain.

From posterior odds to the BIR score (optional reading)

Bayes' rule separates the posterior odds into prior odds and document evidence:

$$\frac{P(R \texttt{ mid } \boldsymbol{d}_i, Q)}{P(NR \texttt{ mid } \boldsymbol{d}_i, Q)} = \frac{P(R \texttt{ mid } Q)}{P(NR \texttt{ mid } Q)} \cdot \frac{P(\boldsymbol{d}_i \texttt{ mid } R, Q)}{P(\boldsymbol{d}_i \texttt{ mid } NR, Q)}. \tag{1.14}$$

The prior odds $P(R \texttt{ mid } Q)/P(NR \texttt{ mid } Q)$ are the same for every document considered for query Q. They affect the numerical score but not the ranking. We can therefore focus on the likelihood ratio containing the document evidence.

Under the binary representation, each component satisfies $d_{i,j} \in \{0, 1\}$: it is 1 when document D_i contains term t_j and 0 otherwise. Combined with term independence, this makes the probability of the complete document vector a product of Bernoulli probabilities. The exponents select the appropriate case: $r_j^{d_{i,j}}$ contributes r_j when the term is present, while $\big(1 - r_j\big)^{1-d_{i,j}}$ contributes $1 - r_j$ when it is absent.

$$P(\boldsymbol{d}_i \texttt{ mid } R, Q) = \prod_{j=1}^{M} r_j^{d_{i,j}} \big(1 - r_j\big)^{1-d_{i,j}}, \tag{1.15}$$

$$P(\boldsymbol{d}_i \texttt{ mid } NR, Q) = \prod_{j=1}^{M} n_j^{d_{i,j}} \big(1 - n_j\big)^{1-d_{i,j}}. \tag{1.16}$$

Substituting these products into the odds and taking the logarithm turns multiplication into addition:

$$\log \frac{P(R \texttt{ mid } \boldsymbol{d}_i, Q)}{P(NR \texttt{ mid } \boldsymbol{d}_i, Q)} = \log \frac{P(R \texttt{ mid } Q)}{P(NR \texttt{ mid } Q)} + \sum_{j=1}^{M} \left[d_{i,j} \log \frac{r_j}{n_j} + \big(1 - d_{i,j}\big) \log \frac{1 - r_j}{1 - n_j} \right]. \tag{1.17}$$

For a term absent from the query, BIR assumes $r_j = n_j$. Both logarithms then become zero, so every non-query term disappears. For a query term, we can rearrange its contribution as

$$\log \frac{1 - r_j}{1 - n_j} + d_{i,j} \log \frac{r_j(1 - n_j)}{n_j(1 - r_j)}. \tag{1.18}$$

The first part depends on the query but not on the document, so it is another constant that does not change the ranking. The second part contributes only when $d_{i,j} = 1$. After removing all ranking constants, the remaining coefficient is

$$c_j = \log \frac{r_j(1 - n_j)}{n_j(1 - r_j)}, \tag{1.19}$$

which produces the additive BIR score below.

Key Formula: BIR Similarity

$$\mathrm{sim}_{\mathrm{BIR}}(Q, D_i) = \sum_{j:\, q_j = 1,\, d_{i,j} = 1} c_j, \qquad c_j = \log \frac{r_j(1 - n_j)}{n_j(1 - r_j)} \tag{1.20}$$

A positive c_j means that term t_j is more characteristic of relevant than non-relevant documents. Documents receive the sum of the weights for query terms they contain.

Relevance Feedback

Initial estimates. Before any relevance information is available, the system must produce a first ranking. BIR commonly assumes that every query term has an equal chance of occurring in a relevant document, $r_j = 0.5$. It estimates occurrence in non-relevant documents from the term's document frequency in the complete collection:

$$r_j = 0.5, \qquad n_j = \frac{\mathrm{df}(t_j) + 0.5}{N + 1}. \tag{1.21}$$

These initial values produce the first c_j weights and therefore the first result list.

Collecting feedback. The user can now mark retrieved documents as relevant or non-relevant, for example through like and dislike controls. These judgements provide a sample of both classes. Because r_j is the probability that term t_j occurs in a relevant document, we can estimate it by counting how many judged relevant documents contain the term. We estimate n_j in the same way from the judged non-relevant documents.

Updating the estimates. Suppose the user has judged K documents and marked L of them as relevant. Let k_j be the number of judged documents containing t_j, and let l_j be the number that both contain t_j and are relevant. The relevant sample therefore contains l_j occurrences among L documents. The non-relevant sample contains $k_j - l_j$ occurrences among $K - L$ documents. Without smoothing, the corresponding estimates would be

$$r_j \approx \frac{l_j}{L}, \qquad n_j \approx \frac{k_j - l_j}{K - L}. \tag{1.22}$$

In practice, BIR adds pseudo-counts so that small samples do not produce probabilities of exactly 0 or 1:

$$r_j = \frac{l_j + 0.5}{L + 1}, \qquad n_j = \frac{k_j - l_j + 0.5}{K - L + 1}. \tag{1.23}$$

The updated probabilities produce new c_j weights and a revised ranking. Further rounds of feedback can refine the estimates, although users may be unwilling to judge many results.

Example: Feedback and query expansion

Consider the query `dog forest`. Suppose all 12 documents are judged, and D_1 and D_3 are marked relevant, leaving the other 10 documents as the non-relevant sample. Both relevant documents contain "dog" and "forest", so feedback first refines the weights of the original query terms:

Term	Relevant documents containing term	Non-relevant documents containing term	r_j	n_j	c_j
"dog"	2	4	0.833	0.409	1.977
"forest"	2	5	0.833	0.500	1.609

With these refined weights, a document containing both "dog" and "forest" scores $1.977 + 1.609 = 3.586$, an improved estimate over the initial, feedback-free weights.

Feedback has a second, independent use: we can compute c_j for terms that never appeared in the query, using the same relevant and non-relevant samples. This tells us which additional terms help separate relevant from non-relevant documents, without requiring the user to reformulate anything.

Term	Relevant documents containing term	Non-relevant documents containing term	r_j	n_j	c_j
"woodland"	2	2	0.833	0.227	2.833
"algorithms"	0	1	0.167	0.136	0.236
"cats"	0	2	0.167	0.227	−0.386

The three terms illustrate three distinct roles:

- **"woodland"** has a high positive c_j: both relevant documents contain it, and only a fifth of the non-relevant documents do. It is a strong candidate for query expansion. Adding it to the query raises D_1 and D_3 to 6.419 and gives lexical variants such as D_2 and D_{11} positive evidence even though they contain neither "dog" nor "forest".
- **"algorithms"** has a c_j close to zero: it occurs in only one document overall, and that document is not relevant. The sample is too small to say whether the term discriminates at all, so it is best discarded, the same treatment we give to stop words that occur everywhere.
- **"cats"** has a negative c_j: it occurs only in non-relevant documents, both of which are technical distractors about pets and animal classification rather than the woodland adventure the user wants. Note that "cats" is a distinct token from "cat", which does appear in D_1; the tokenizer does not know they share a root, so the two counts are entirely independent. A negative weight can therefore help demote or eliminate documents that share surface vocabulary with the query but belong to the wrong sense.

In practice, an automatic query expansion step keeps terms with a strongly positive c_j, drops terms near zero, and may use strongly negative terms to penalize rather than reward a document.

BIR marks a change in ambition, not only a change in formula. The Vector Space Model ranks by geometric heuristics that happen to work well; BIR instead tries to *explain* relevance from first principles, estimating how likely a document is to be relevant given its terms. It still represents documents as term vectors and still combines evidence with the same OR-like accumulation as VSM: any query term found in the document contributes its weight to the score. What changes is the meaning of that weight. Instead of a heuristic tf-idf product, c_j estimates the independent contribution of each query term toward relevance, grounded in how often the term separates relevant from non-relevant documents.

The restrictive assumptions behind this first version, binary term presence, term independence, and no document-length effect, are not inherent to the probabilistic idea itself. They make BIR tractable, and later probabilistic models relax them one at a time. The 2-Poisson model, for instance, models within-document term frequency directly instead of treating term presence as binary, at the cost of a more complex parameter estimation problem; see Robertson and Walker's **Some Simple Effective Approximations to the 2-Poisson Model for Probabilistic Weighted Retrieval** in the Further Reading section of the

chapter summary. Probabilistic language models take yet another route, estimating the probability that a document's language model would generate the query. The section below develops BM25, which keeps BIR's additive, per-term structure but approximates the 2-Poisson model's term-frequency behaviour with a simpler saturating function.

The feedback mechanism also has a practical limitation worth mentioning. A user who rates a first result list of 10 documents gives us a sample of size $K = 10$ from which to estimate r_j and n_j, far too small to pin down these probabilities reliably, especially for terms that appear in only one or two of the judged documents. Larger judged samples would improve the estimates, but no user will realistically rate hundreds of documents to make search work. This tension between statistical need and user effort is a recurring theme in relevance feedback, not a flaw specific to BIR.

Okapi BM25

BM25 was developed at London's City University by Stephen Robertson, Karen Spärck Jones, and colleagues as a practical approximation to the 2-Poisson model. It became the robust default for free-text retrieval that BIR itself never was. Robertson and Zaragoza's **The Probabilistic Relevance Framework: BM25 and Beyond**, listed in the Further Reading section of the chapter summary, surveys the complete derivation and its later extensions.

> **What is the meaning of "Okapi BM25"?**
>
> The name comes from Okapi, the experimental retrieval system built at City University London in the 1980s and 1990s where the formula was first implemented, not from any property of the ranking function itself. "BM" stands for "Best Matching", and "25" simply identifies it as the 25th weighting variant the researchers tried.

Three Unresolved Issues

Before introducing the BM25 formula, it helps to see exactly what the two preceding models still get wrong. Each issue below reappears as a concrete design decision in BM25.

Vector-space scoring rewards repetition without limit. Recall the running query `cat dog forest` from the Vector Space Retrieval section. Its inner product already ranks D_7 (four occurrences of "dog", score 2.773 for that term alone) above D_9 (one occurrence of each query term, contributing 0.693 for "dog" toward its combined score of 1.537), even though D_9 is the only document that mentions "cat", "dog", and "forest" together. If a document repeated "dog" twenty times instead of four, its score would climb to 13.863, growing linearly and without bound. A ranking function that rewards raw repetition this directly is vulnerable to keyword stuffing: an author can inflate a document's score simply by repeating a term, regardless of whether the document is actually about that topic.

Raw term frequency ignores what "coverage" means for document length. A term occurring once in a 3-token document (such as D_9) is stronger evidence of relevance than the same term occurring once in a 20-token document, because in the short document, that one occurrence forms a much larger share of the content. TF-IDF and the inner product treat both occurrences identically. Relevance should depend on how much of the document is about the query term, not merely how many times it appears. To reward the same relevance signal, a longer document should need proportionally more occurrences of a term than a shorter one to reach the same score.

Binary presence throws away frequency information, but the idea behind r_j and n_j is worth keeping. BIR's document evidence is either 0 or 1, so it cannot express that D_7 mentions "dog" four times while D_3 mentions it once. At the same time, BIR contributes something valuable: the idea that a term's ranking weight should reflect how well that term distinguishes relevant from non-relevant documents, not just how rare it is in the collection overall. BM25 keeps this probabilistic idea but combines it with a real-valued term-frequency component.

Term-Frequency Saturation

BM25 replaces the raw count in the TF-IDF product with a bounded function of term frequency:

$$\hat{\mathrm{tf}}_{k_1} = \frac{\mathrm{tf}(k_1+1)}{\mathrm{tf}+k_1}, \qquad k_1 > 0. \tag{1.24}$$

The first occurrence contributes strongly; later occurrences add progressively less; the value never exceeds $k_1 + 1$ no matter how often the term repeats. This directly answers the spamming issue: repeating "dog" twenty times can no longer produce an unbounded score. Parameter k_1 controls how quickly the function saturates. Values between 1 and 2 are common, with $k_1 = 1.2$ used in the running example. Figure 1.15 contrasts linear, square-root, and saturating term-frequency weights.

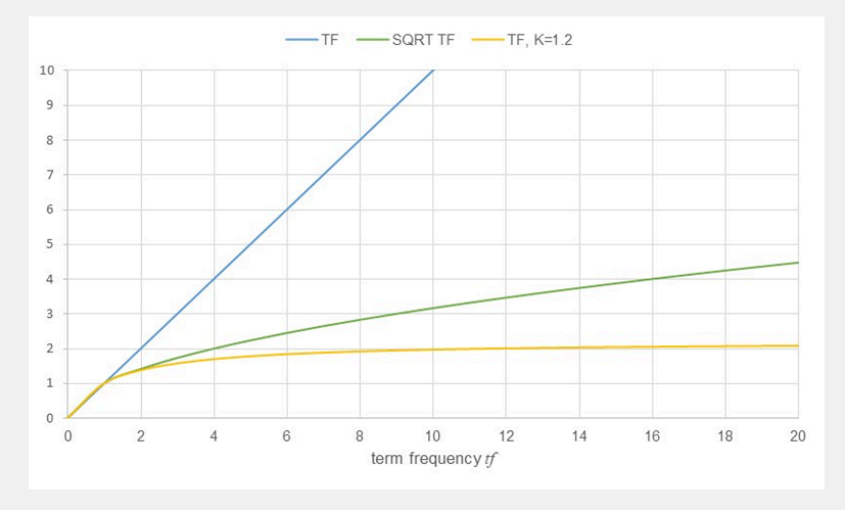


Figure 1.15: Linear, square-root, and BM25-saturated term-frequency weighting.

Document-Length Normalization

Saturation alone does not address the length issue: a document with tf=1 in 3 tokens and a document with tf=1 in 20 tokens still saturate identically. BM25 makes the saturation point depend on document length by scaling the denominator:

$$\hat{\mathrm{tf}}_{k_1,b}(D_i,t) = \frac{\mathrm{tf}(D_i,t)(k_1+1)}{\mathrm{tf}(D_i,t) + k_1\left(1-b+b\frac{|\,D_i\,|}{\mathrm{avgdl}}\right)}. \tag{1.25}$$

Here $|\,D_i\,|$ is the number of processed tokens (=document length) and avgdl is the collection's average document length. For the running collection, avgdl $= 9.67$ tokens. A single occurrence of "dog" in a 3-token document (shorter than average) contributes 1.393, while the same single occurrence in a 20-token document (longer than average) contributes only 0.696: the short document needs less repetition to reach the same weight because that occurrence covers a larger share of its content. Parameter $b \in [0,1]$ controls the strength of this effect: $b = 0$ disables length normalization entirely, while $b = 1$ applies the full length ratio. Figure 1.16 shows how the same term frequency receives more weight in a short document than in a long one.

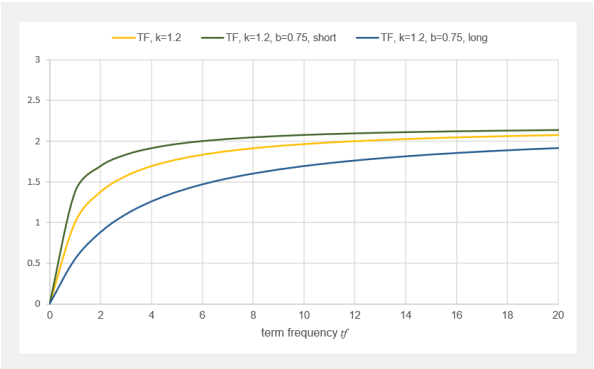


Figure 1.16: BM25 term-frequency saturation for average, short, and long documents.

Probabilistic IDF

The third issue was that BIR's idea of estimating a term's discriminating power from r_j and n_j is worth keeping even though its binary document model is not. Substituting the no-feedback BIR estimates ($r_j = 0.5$ and n_j from document frequency) into the c_j formula from the previous section gives exactly the classical BM25 weight:

$$\mathrm{idf}_{\mathrm{BM25}}(t) = \log \frac{N - \mathrm{df}(t) + 0.5}{\mathrm{df}(t) + 0.5}. \tag{1.26}$$

This value becomes negative when a term appears in more than half of the collection. Many implementations instead use the always-positive Lucene variant

$$\mathrm{idf}_{+}(t) = \log\left(1 + \frac{N - \mathrm{df}(t) + 0.5}{\mathrm{df}(t) + 0.5}\right) = \log \frac{N + 1}{\mathrm{df}(t) + 0.5}. \tag{1.27}$$

Figure 1.17 compares these variants with the smoothed classical IDF introduced earlier. All encode the same core intuition that common terms provide less evidence, but their numerical values are not interchangeable.

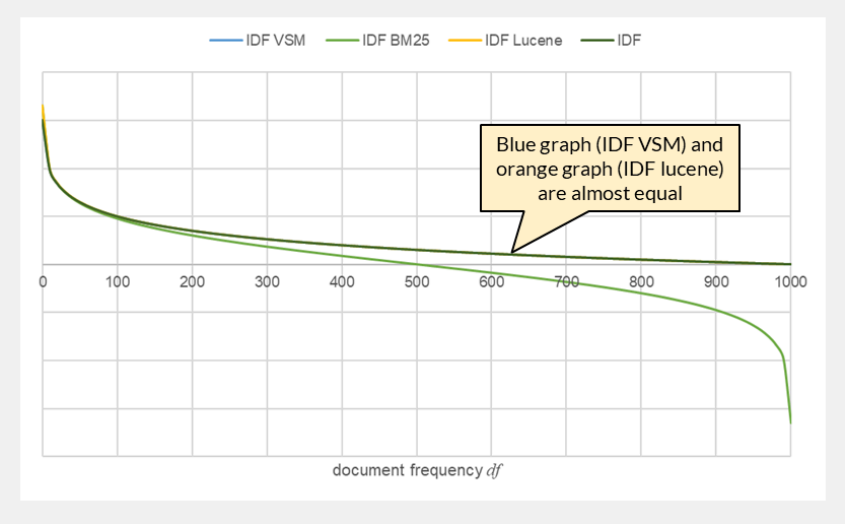


Figure 1.17: Classical, original BM25, and positive Lucene IDF variants.

A negative weight is undesirable for an additive scoring function: it would let a matching query term actively lower a document's score, the opposite of what a match should do, and it complicates summing evidence across query terms of very different commonness. The Lucene weight $\log \frac{N+1}{\mathrm{df}(t)+0.5}$ and the smoothed classical VSM weight $\log \frac{N}{\mathrm{df}(t)}$ are nearly identical: comparing them shows that the Lucene variant is equal to the classical one exactly when $\mathrm{df}(t) = N/2$, slightly below it for rare terms, and slightly above it for common terms, never departing far in either direction. This is exactly the region where the original BM25 IDF turns negative, so Lucene's variant tracks the familiar classical shape almost everywhere while staying non-negative where it matters. Lucene therefore adopts it as a numerically safer stand-in for the same underlying quantity rather than as a conceptually different weighting scheme.

Note

The standard BM25 formula uses the no-feedback estimate for r_j and does not itself perform a feedback step. Nothing prevents combining BM25 with relevance feedback: a system can substitute the feedback-refined r_j and n_j from BIR into the same weight instead of the fixed $r_j = 0.5$. In practice, systems built on Lucene use the fixed no-feedback form as the default.

Complete Formula and Running Example

Using the positive IDF variant gives the practical scoring function used in this example.

Key Formula: BM25

$$\mathrm{sim}_{\mathrm{BM25}}(Q, D_i) = \sum_{t \in Q \cap D_i} \mathrm{idf}_+(t) \frac{\mathrm{tf}(D_i, t)(k_1 + 1)}{\mathrm{tf}(D_i, t) + k_1 \left(1 - b + b \frac{|D_i|}{\mathrm{avgdl}}\right)} \tag{1.28}$$

BM25 sums query-term evidence after applying probabilistic term specificity, diminishing returns for repetition, and document-length normalization.

For $Q =$ `cat dog forest`, use $k_1 = 1.2$, $b = 0.75$, and the same preprocessing as before. The 12 documents have avgdl $= 9.67$ tokens. Their positive IDF values are 0.860 for "cat", 0.693 for "dog", and 0.550 for "forest".

Example: BM25 ranking

Document	Length	TF (`cat`, `dog`, `forest`)	BM25 score
D_9	3	(1, 1, 1)	2.930
D_1	8	(2, 2, 1)	2.836
D_{10}	14	(2, 2, 2)	2.568
D_{12}	11	(1, 1, 0)	1.470
D_3	10	(0, 1, 1)	1.226
D_7	10	(0, 4, 0)	1.166
D_4	8	(1, 0, 0)	0.925
D_8	12	(0, 0, 4)	0.894
D_5	11	(0, 0, 2)	0.728
D_6	10	(0, 0, 1)	0.542

Documents D_2 and D_{11} share no exact query term and score zero. The compact exact match D_9 ranks first, resolving the ordering that VSM's inner product got wrong. Repetition keeps D_1 competitive, but saturation prevents the four occurrences of "dog" in D_7 from dominating the way it did under linear TF-IDF. Although D_{10} repeats all three terms, its greater length lowers its score relative to D_9: the same occurrences carry less weight once length normalization is applied.

Evaluation follows the same efficient pattern as vector-space retrieval. The system takes the union of the query-term posting lists, accumulates one BM25 contribution per matching term, and sorts candidates by decreasing score. Chapter 4 develops the posting-list algorithms used for this process.

Limitations and Modern Applications

BM25 remains lexical and bag-of-words based. It neither connects "cat" with "feline" nor distinguishes the natural and technical meanings of "forest". It also assumes that document length should influence relevance in a consistent way, so k_1 and b may require tuning for collections with unusual document types. Despite its probabilistic derivation, a BM25 score is a ranking value, not a calibrated probability of relevance.

Advantages: BM25 combines partial matching, discriminative term weighting, term-frequency saturation, and document-length normalization in an efficient additive score. It is transparent, fast, and difficult to improve upon as a lexical baseline.

Disadvantages: It retains lexical matching and term-independence assumptions, requires parameter choices, and does not directly represent phrase meaning or semantic similarity.

BM25 is the default or standard lexical ranker in systems built on Lucene, including Elasticsearch, Solr, and OpenSearch. It is also widely used as a first-stage retriever in retrieval-augmented generation and in hybrid search, where its exact lexical matches complement dense embedding retrieval.

Hands-on: Retrieval Models

> Implement Boolean, TF-IDF, and BM25 retrieval on a small collection and compare their rankings.
>
> Open notebook ->

Summary

Model Comparison

Property	Boolean	Extended Boolean	Vector Space	BIR	BM25
Query form	Boolean expression	Boolean expression	Free text	Term set	Free text
Ranked output	No	Yes	Yes	Yes	Yes
Partial matches	No	Yes	Yes	Yes	Yes
Term frequency	Ignored	Normalized TF-IDF	Linear TF-IDF	Ignored	Saturating
Document length	Ignored	Indirect normalization	Cosine or none	Ignored	Explicit parameter b
Foundation	Set theory	Soft-logic heuristic	Geometric heuristic	Relevance probability	BIR plus TF and length models
Typical role	Exact filters	Specialized soft constraints	Baseline and vector similarity	Feedback model and foundation	Production lexical ranking
Main weakness	No ranking	Operator choices	Length and repetition effects	Binary evidence and feedback burden	Lexical matching and tuning

The progression is cumulative rather than a sequence of complete replacements. Boolean predicates remain useful for mandatory filters, vector operations remain central to dense retrieval, BIR explains relevance feedback and probabilistic term weights, and BM25 provides the strongest general-purpose lexical ranking baseline.

Key Takeaways

1. Text retrieval represents documents and queries using the same vocabulary, then measures similarity to estimate relevance — this direct word-level matching makes text the least affected modality by the semantic gap.
2. The feature extraction pipeline (Extract arrow.r Split arrow.r Tokenize arrow.r Summarize) transforms raw text into searchable representations. The bag-of-words model records term frequencies; the set-of-words model records only term presence.
3. IDF weighting captures term discrimination power: rare terms that appear in few documents are more informative for distinguishing relevant from non-relevant results.
4. Retrieval models evolved from Boolean (no ranking, exact match) through the Extended Boolean Model (partial matches, simple scoring) to the Vector Space Model (tf-idf, cosine similarity) and Probabilistic Retrieval (BIR model, relevance feedback).
5. BM25 combines the best ideas: saturating term frequency, document-length normalization, and probabilistically grounded IDF — it remains the default baseline in modern search engines (Lucene, Elasticsearch, OpenSearch).

Key Formulas

Key Formula: TF-IDF

$$\text{tf-idf}(t, d) = \text{tf}(t, d) \cdot \log \frac{N}{\text{df}(t)} \tag{1.29}$$

Term frequency × inverse document frequency. Rewards terms that are frequent in a specific document but rare across the collection.

Key Formula: Cosine Similarity

$$\text{sim}(\boldsymbol{q}, \boldsymbol{d}) = \frac{\boldsymbol{q} \cdot \boldsymbol{d}}{|\,\boldsymbol{q}\,| \cdot |\,\boldsymbol{d}\,|} \tag{1.30}$$

Measures the angle between query and document vectors, normalizing for document length.

Key Formula: BM25

$$\text{sim}_{\text{BM25}}(Q, D) = \sum_{t \in Q \cap D} \log\left(1 + \frac{N - \text{df}(t) + 0.5}{\text{df}(t) + 0.5}\right) \cdot \frac{\text{tf}(t, D) \cdot (k_1 + 1)}{\text{tf}(t, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|\,D\,|}{\text{avgdl}}\right)} \tag{1.31}$$

Probabilistic term specificity multiplied by saturating, length-normalized term frequency. Parameter k_1 controls saturation, b controls length normalization, and avgdl is the collection's average document length.

Exam focus

Beyond the formulas above, be able to explain:

- Why the Standard Boolean Model cannot rank documents, and what the Extended Boolean Model changes to enable graded scores.
- Why the inner product favors long, repetitive documents while cosine similarity favors documents whose term proportions match the query, and why cosine can still penalize relevant content.
- Why BIR's binary term-presence assumption is too coarse, and how relevance feedback estimates r_j and n_j.
- Why the classical, BM25, and Lucene IDF variants are not numerically interchangeable, even though they encode the same intuition.
- How BM25's saturation parameter k_1 and length-normalization parameter b each address a specific weakness of TF-IDF scoring.

Self-Check Questions

1. (Understand) Explain in your own words why the Boolean model cannot rank documents, and what the Extended Boolean Model changes to enable ranking.
2. (Analyze) Given a collection of 10,000 documents where the term "retrieval" appears in 50 documents and the term "the" appears in 9,800 documents, calculate the IDF for both terms. Why does this difference matter for relevance scoring?
3. (Analyze) A document is 3× longer than the average document in the collection. How does BM25 (with $b = 0.75$) adjust the effective term frequency compared to an average-length document? What happens when $b = 0$?
4. (Evaluate) An e-commerce search system currently uses Boolean retrieval with faceted filtering. A developer proposes switching to BM25. What are the trade-offs? In which scenarios might Boolean retrieval still be preferable?

Test Your Knowledge

Take the Chapter 1 Quiz –>

Further Reading

- Spärck Jones, K. (1972). **A Statistical Interpretation of Term Specificity and Its Application in Retrieval**. *Journal of Documentation*, 28(1), 11–21. PDF — Introduced inverse document frequency; foundational for every term-weighting scheme discussed in this chapter, including TF-IDF and BM25.
- Salton, G., Wong, A., & Yang, C. S. (1975). **A Vector Space Model for Automatic Indexing**. *Communications of the ACM*, 18(11), 613–620. PDF — The original paper introducing the Vector Space Model; still readable and surprisingly concise.
- Robertson, S. E., & Spärck Jones, K. (1976). **Relevance Weighting of Search Terms**. *Journal of the American Society for Information Science*, 27(3), 129–146. Publisher page — Formalizes the Binary Independence Model and its r_j/n_j relevance-feedback estimates covered in this chapter.
- Salton, G., Fox, E. A., & Wu, H. (1983). **Extended Boolean Information Retrieval**. *Communications of the ACM*, 26(11), 1022–1036. Free access — Introduces the p-norm model, positioning graded Boolean retrieval between the strict Boolean and vector-space models.
- Porter, M. F. (1980). **An Algorithm for Suffix Stripping**. *Program*, 14(3), 130–137. Author's page — The classic stemming algorithm still used as a baseline in English text processing.
- Robertson, S. E., & Walker, S. (1994). **Some Simple Effective Approximations to the 2-Poisson Model for Probabilistic Weighted Retrieval**. In *Proceedings of the 17th ACM SIGIR Conference*, 232–241. PDF — Derives BM25′s term-frequency saturation and length normalization from the 2-Poisson model discussed in the Probabilistic Ranking and BM25 section.
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). **Introduction to Information Retrieval**. Cambridge University Press. Free online edition — The standard graduate textbook; chapters 1–6 cover Boolean, VSM, and probabilistic models with rigorous treatment.
- Robertson, S. E., & Zaragoza, H. (2009). **The Probabilistic Relevance Framework: BM25 and Beyond**. *Foundations and Trends in Information Retrieval*, 3(4), 333–389. PDF — Definitive survey of BM25′s theoretical foundations and practical extensions.